UNIVERSITÉ ABDELMALEK ESSAADI

FACULTÉ DES SCIENCES ET TECHNIQUES DE TANGER

Département Génie Informatique

Mémoire de Projet de Fin d'Études

MASTER SCIENCES ET TECHNIQUES EN SÉCURITÉ IT & BIG DATA

SUJET

Learning Offensive Navigation Policies on Heterogeneous Attack Graphs

RÉALISÉ PAR : SENBATI NIZAR

SOUTENUE LE 02/07/2026 DEVANT LE JURY

Pr. Lotfi EL AACHAK	PRÉSIDENT
Pr. Abdelhadi FENNAN	EXAMINATEUR
Pr. Abderrahim GHADI	ENCADRANT FSTT
Mr. ZAABOUL Jawad	ENCADRANT ENTREPRISE

Année Universitaire 2025/2026

Acknowledgements

First and foremost, I would like to express my deepest gratitude to my academic supervisor GHADI Abderahim, for his unwavering guidance, patience, and invaluable feedback throughout this academic year.

I am equally grateful to my professional supervisor, ZAABOUL Jawad, and the entire team at NearSecure. Thank you for providing an environment that fosters innovation, for trusting me to pursue the intersection of offensive security and Deep Learning, and for the technical mentorship that bridged the gap between academic theory and real-world application.

A massive thank you is owed to the broader cybersecurity and open-source communities. The development of the GNN-AD-Navigator would not have been possible without the tireless work of the developers who build and maintain the tools we rely on. Specifically, I would like to acknowledge:

SpecterOps, for creating and open-sourcing BloodHound, which provided the foundational graph topology and data schema necessary for my model's ingestion pipeline.

Oliver Lyak ly4k, for developing Certipy, an essential tool that allowed me to accurately map complex Active Directory Certificate Services ADCS attack paths into my heterogeneous graph.

Mayfly Orange Cyberdefense, for creating GOAD Game of Active Directory. The immense complexity of this lab environment served as the perfect training ground for the neural network, allowing it to learn and discover emergent tactical behaviors.

I also want to thank Hack The Box Academy for their student program, which granted me accessible access to the inlanefreight Active Directory environment. This provided the critical cross-environment validation required to prove the model's efficacy outside of its training data.

Finally, a personal thank you to my father and familly for the ongoing support, advice and guidance during the long hours spent debugging and studying. To everyone who supported me through this Master's journey: thank you.

Abstract

Active Directory is a directory service so deeply embedded in corporate infrastructure that misconfigurations, no matter how small, can expose entire organisations to privilege escalation attacks. Tools like BloodHound are critical for both defenders and red teams: they map the environment as a graph and let operators query attack paths between any two objects. BloodHound's built-in pathfinding, however, runs plain Dijkstra — it returns the shortest path, not necessarily the most operationally sound one, and it cannot learn from prior engagements.

This thesis asks whether a Graph Neural Network can learn a navigation policy from labelled attack traces and generalise it to entirely unseen enterprise environments. A Heterogeneous Graph Transformer is trained on a multi-domain lab environment and evaluated on a 4 000-node enterprise network it has never seen. The model recovers the expert-documented Domain Admin attack path without retraining, establishing cross-environment generalisation as a proof of concept. Identified architectural boundaries and a forward-looking hypothesis for a second-generation tool are discussed.

Keywords: Graph Neural Network, Heterogeneous Graph Transformer, Active Directory, Privilege Escalation, Attack Graph, Beam Search, Offensive Security, Cross-Environment Generalisation, Active Directory Cerfication Services.

Résumé

Active Directory est un service d'annuaire si profondément intégré dans l'infrastructure des entreprises que la moindre erreur de configuration peut exposer une organisation entière à des attaques de privilege escalation. Des outils comme BloodHound sont essentiels à la fois pour les défenseurs et les red teams : ils cartographient l'environnement sous forme de graphe et permettent aux opérateurs de rechercher des chemins d'attaque entre deux objets. Cependant, le moteur de recherche de BloodHound utilise le simple algorithme de Dijkstra renvoie le chemin le plus court, et non le plus pertinent d'un point de vue opérationnel.

Cette thèse cherche à savoir si un Graph Neural Network peut apprendre une politique de navigation à partir de traces d'attaques étiquetées, et la généraliser à des environnements d'entreprise totalement inconnus. Un Heterogeneous Graph Transformer est entraîné sur un environnement de laboratoire multi-domaines, puis évalué sur un réseau d'entreprise de 4 000 nœuds qu'il n'a jamais vu auparavant. Le modèle parvient à retrouver le chemin d'attaque Domain Admin documenté par des experts, et ce sans réentraînement, validant ainsi la généralisation cross-environment comme un véritable Proof of Concept. Enfin, les limites architecturales identifiées ainsi que des pistes d'évolution pour un outil de seconde génération sont discutées.

Mots-clés : Réseau de neurones sur graphe, Transformateur de graphe hétérogène, Active Directory, Escalade de privilèges, Graphe d'attaque, Recherche en faisceau, Sécurité offensive, Généralisation inter-environnements.

Contents

List of Figures

List of Tables

List of Abbreviations

ACE Access Control Entry. 9

ACL Access Control List. 9, 21, 23, 35, 48, 52, 55

AD Active Directory. 18, 19, 22, 25, 27, 29, 35, 36, 40, 56

ADCS Active Directory Certificate Services. 2, 12, 18, 21–23, 30, 33, 40, 52, 53, 60, 61, 66

APT Advanced Persistent Threat. 22

CA Certificate Authority. 22

CLI Command Line Interface. vii, 36–38, 74

CVE Common Vulnerabilities and Exposures. 35

DCSync Directory Replication Service synchronisation attack. 28, 29, 35, 48, 55, 63, 65

EDR Endpoint Detection and Response. 9

GCN Graph Convolutional Network. 15

GNN Graph Neural Network. 10, 13, 21, 22, 62, 64

GOAD Game of Active Directory. 2, 15, 34, 40, 42, 45, 52, 54, 55, 65

GPO Group Policy Object. 21, 40

GPU Graphical Processing Unit. 31

GUI Graphical User Interface. 36, 38, 46, 65

HGT Heterogeneous Graph Transformer. 2, 15, 25–27, 29, 30, 33, 35, 36, 38, 40, 42, 49, 51, 54, 57, 58, 61–63, 65, 69, 75

LDAP Lightweight Directory Access Protocol. 52

LotL Living of the Land. 27, 58, 62, 64

MDI Microsoft Defender for Identity. 9

MLP Multi-Layer Perceptron. 26, 54

NTLM NT LAN Manager. 22

OOV Out Of Vocabulary. 27, 63

. 57, 62

OPSEC Operational Security. 9, 10

OU Organisational Unit. 21, 22, 61

PyG PyTorch Geometric. 2, 18, 25, 33, 35, 46, 52, 56, 65, 69

RBCD Resource-Based Constrained Delegation. 2, 35

ReLU Rectified Linear Unit. 26

RL Reinforcement Learning. 63

RPC Remote Procedure Call. 40, 52

SID Security Identifier. 2, 21, 22, 37, 46, 47

SIEM Security Information and Event Management. 9, 58

smb Server Message Bloc. 40

General Introduction

Context

Active Directory is the identity backbone of major enterprises. While typically isolated withing the inside network, once the perimeter defenses are breached (e.g., via web service exploitation, phishing, or a compromised usb) an advarsary gains the ability to directly interacte and query the Active Directory Domain Controller. Through protocols such as LDAP and SMB the entirety of the organisation's IT infrastructure , data and services is essencialy mapped out for the attacker. A vulnerable AD poses extreme levels of risk making hardening the AD network environment an extreme priority.

Once inside, attackers frequently leverage credential-based and ACL-based attacks such as pass-the-hash, DCSync, Kerberoasting are among the most impactful attack vectors in modern breaches. Red teams and security auditor alike rely on tools like BloodHound to map the attack surface and identify the vulnerable nodes during an engagement. However the analysis remains a highly manual process, time consuming, heavely depandent on human expertise and none scalable in large enterprise environments.

Problem Statement

Bloodhound maps the entirety of the Active Directory to a graph database, each node represents an object, and the relationships between these objects is represented by edges. When a user runs a path finding query, BloodHound simply runs Dijkstra over the attack graph, it finds the shortest path, not necessarily the most realistic or discreet one.
The question this thesis asks : can a model learn a navigation policy and generalise that policy to an unseen entreprise environment?

Objectives

The objectif of this Thesis is to provide a proof of concept for learned attack path discovery over heterogeneous Active Directory graphs.
To that end this work aims to:

- Build an end-to-end pipeline transforming raw BloodHound and Certipy JSON

to navigable PyTorch Geometric (PyG) `HeteroData` tensors.

- Train a GNN-based navigation policy and demonstrate cross-environment generalisation to unseen entreprise level environments.
- Identify and precisely describe the model's limitations and architectural boundaries through systematic evaluation, establishing future work and improvements.

Industrial Context: NearSecure

This research was conducted in collaboration with **NearSecure**, a specialized cybersecurity firm providing advanced offensive security assessments and Red Team operations. During enterprise engagements, security consultants frequently encounter massive, highly complex Active Directory environments. While tools like BloodHound are standard for mapping these networks, NearSecure identified a critical operational bottleneck: the manual analysis of large-scale graph data is incredibly time-consuming and heavily reliant on the subjective expertise of the individual pentester. Furthermore, traditional shortest-path queries fail to account for Operational Security (OPSEC), often highlighting "loud" attack vectors that immediately trigger Blue Team telemetry.

To address this industry challenge, this thesis project was initiated to bridge the gap between deep learning and offensive security. The objective was to engineer the `GNN-AD-Navigator`—an automated, AI-driven pipeline capable of augmenting NearSecure's Red Team capabilities by surfacing not just the shortest, but the most operationally viable and stealthy attack paths.

Main Contributions

- A complete, modular heterogeneous graph pipeline from raw BloodHound/Certipy JSON to navigable PyG `HeteroData` tensors, covering Active Directory Certificate Services (ADCS), Resource-Based Constrained Delegation (RBCD), GPLink, and Security Identifier (SID) history edges.
- A Heterogeneous Graph Transformer (HGT)-based navigation policy trained on Game of Active Directory (GOAD) traces and evaluated on the entirely unseen INLANEFREIGHT enterprise environment, demonstrating generalisation from a 351-node lab to a 4 000-node network.
- An evaluation of the tool's current limitations. Identifying how the search algorithm naturally penalizes long paths, which creates an unintended bias toward shorter, noisier attacks rather than longer, stealthier desired routes. A highlight of the model's limited line of sight receptive field when trying to discover deep,

multi-step continuous attack chains. Also acknowledging that testing was restricted to lab environments rather than enterprise networks due to time and resource constraints.

- A proposal for a future, second-generation tool that addresses these limitations by: implementing a guided search algorithm (such as A-Star) to help the model value stealth and keep long attack paths alive; using natural language AI to read and understand entirely new types of attacks based on their descriptions instead of relying on a hardcoded list; and adding a time dimension to the graph to accurately reflect temporary network states, such as active user sessions that log on and off. An automated "divide-and-conquer" routing system that breaks excessively long attack chains into smaller manageable segments using major network hubs or domain boundaries as checkpoints.

Report Outline

The remainder of this document is structured as follows.

Chapter 1 establishes the theoretical foundation, reviewing the architecture of Active Directory, common privilege escalation techniques, the machine learning logic behind GNNs, and the current state of graph-based attack path analysis.

Chapter 2 details the methodology and system architecture, introducing the heterogeneous graph pipeline designed to transform raw BloodHound data into navigable tensors. It also presents the core logic of the navigation policy, the loss function, and the beam search method.

Chapter 3 presents the internal architecture and the training paradigm of the Graph Neural Network GNN navigation policy.

Chapter 4 provides an empirical evaluation of the learned policy across multiple environments, documenting both successful generalizations and precise failure modes. It evaluates the tool's current limitations, specifically identifying how the search algorithm naturally penalizes long paths (creating a bias toward shorter, noisier attacks) and defining the model's limited receptive field when navigating deep attack chains.

Chapter 5 concludes the report by reviewing the achievements of this proof of concept and honestly assessing its limitations. It then proposes a second-generation tool featuring practical engineering improvements, such as automated waypoint routing to chunk long paths, guided search algorithms to prioritize stealth, and the integration of natural language models to dynamically interpret novel attack techniques.

Chapter 1

Background and Related Work

1.1 Active Directory: Architecture and Security Model

1.1.1 Core Objects and Structure

Active Directory (AD) a directory service for Windows network environments. It is a distributed, hierarchical structure that allows for centralized management of an organization's resources, including users, computers, groups, network devices, file shares, group policies, servers and workstations, and trusts. AD provides authentication and authorization functions within a Windows domain environment.[1]

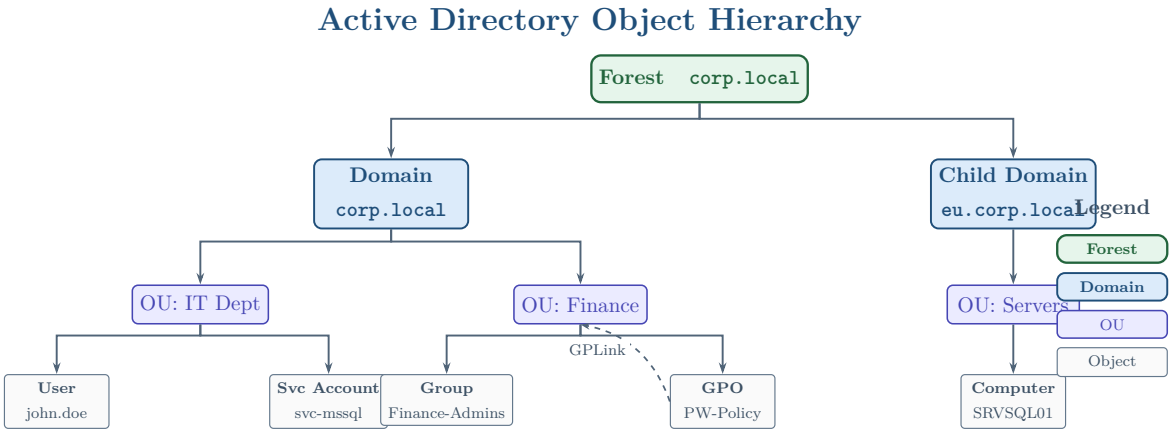

Figure 1.1: Active Directory object hierarchy.

Figure 1.1 illustrates the standard hierarchical structure of an Active Directory environment. At the highest level is the Forest, which serves as the absolute security boundary for the organization. Within the forest, one or more Domains act as logical partitions to manage resources. To streamline administration, these domains are further subdivided into Organizational Units OUs. OUs act as geographical or functional containers to group specific base objects, such as users, service accounts, security groups, and computers. Additionally, the diagram highlights how Group Policy Objects GPOs can be linked directly to OUs to enforce specific security configurations across all objects

within that container.

By design, Active Directory allows almost any authenticated user regardless of their administrative privilege level to read and query the directory. This open read-access is intended to facilitate normal network operations, such as allowing a user to look up a colleague's email address or locate a network printer. However, this feature creates a significant security vulnerability.

If an adversary compromises a single, unprivileged user account or workstation, they can abuse this read access using standard protocols like LDAP and RPC. The attacker can silently sweep the directory to extract the exact same list of computers, group memberships, trust relationships, and Access Control Lists (ACLs) detailed above. By stitching this data together, the adversary can map the entire network topology and identify hidden, multi-step privilege escalation paths leading from their starting point to high-value targets like Domain Admin.

Because the relationships between thousands of Active Directory objects create a highly complex web of permissions, manually identifying these chained vulnerabilities is nearly impossible. To address this, security professionals, defenders, and penetration testers utilize graph-based pathfinding tools. By ingesting the same directory data that an attacker would enumerate, these tools model the Active Directory environment as a mathematical graph. This allows operators to visualize hidden attack paths, proactively identify structural weaknesses, and remediate dangerous permission chains before they can be exploited during a real breach.

A standard Active Directory user account with no administrative privileges can be leveraged to query and enumerate a vast amount of structural and security data across the network. This accessible information includes, but is not limited to:

- Domain Computers
- Domain Group Information
- Default Domain Policy
- Password Policy
- Domain Trusts
- Domain Users
- Organizational Units (OUs)
- Functional Domain Levels
- Group Policy Objects (GPOs)
- Access Control Lists (ACLs)

1.1.2 Authentication Protocols

Active Directory manages network security through the fundamental AAA framework: Authentication, Authorization, and Accounting [2]. When a user, machine, or service

attempts to access a network resource, AD first verifies their cryptographic identity (Authentication). Once identity is proven, the system evaluates the user's embedded Security Identifier (SID) and group memberships against the target resource's Access Control Lists (Authorization). Finally, the Domain Controller logs the logon events and access requests (Accounting) to ensure auditability. To execute the initial authentication phase, Active Directory relies primarily on two distinct network protocols: Kerberos and NTLM.

Kerberos

Kerberos is the default and preferred authentication protocol in modern Active Directory environments [3]. It operates as a sophisticated, ticket-based system brokered by a Key Distribution Center (KDC), a service that runs natively on every Domain Controller. Rather than transmitting passwords or hashes across the network, Kerberos relies on symmetric-key cryptography. Upon a successful initial logon, the KDC issues the client a Ticket Granting Ticket (TGT). When the client needs to access a specific network service (such as a file share or database), it presents the TGT to the KDC to request a Service Ticket (ST) for that specific resource. While Kerberos relies on strict timestamping to prevent replay attacks and is cryptographically robust, its complex delegation and ticketing mechanics make it a primary target for advanced adversaries who abuse it via techniques such as Pass-the-Ticket, Golden Tickets, and delegation hijacking.

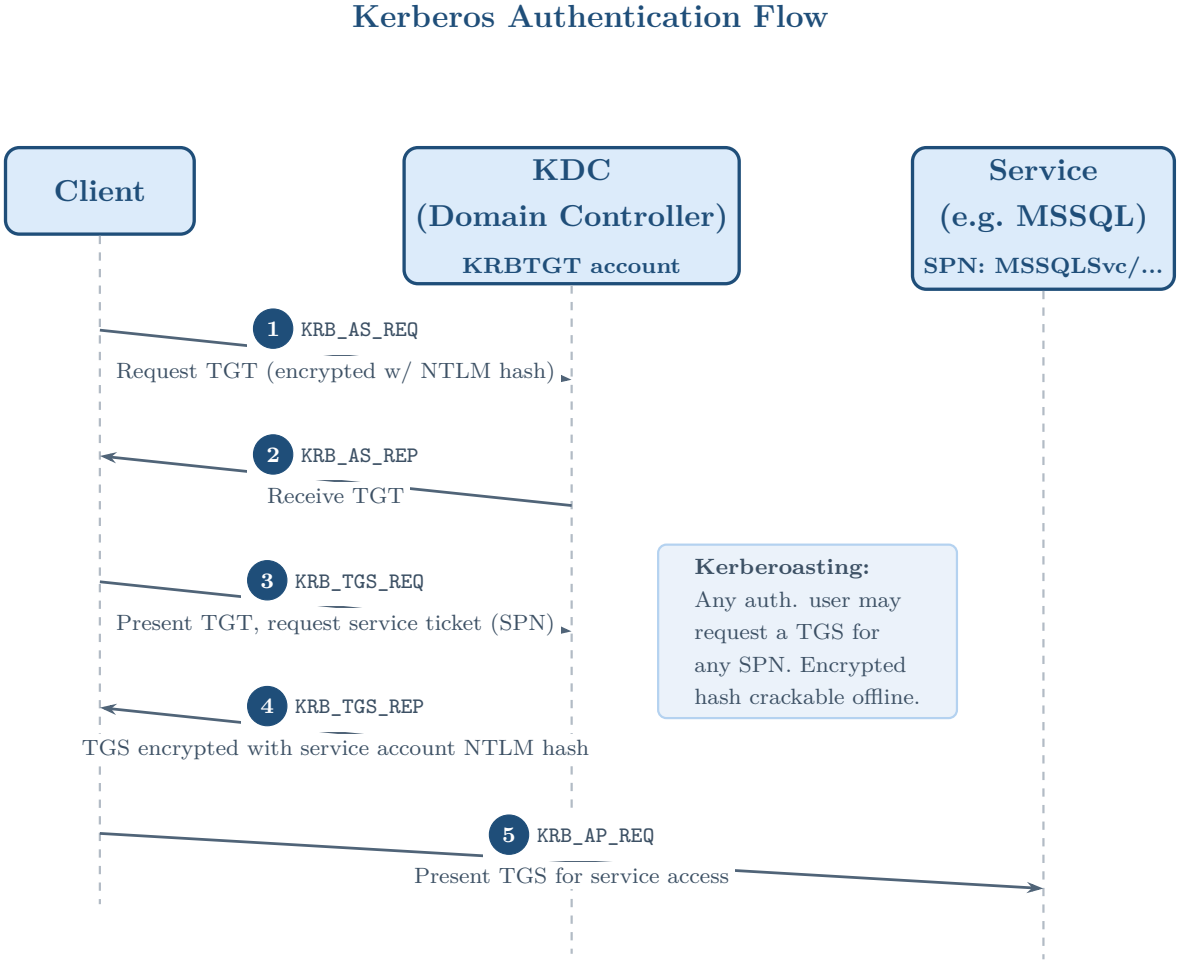

Figure 1.2: Kerberos Authentication Flow

Within the specific context of this thesis, two Kerberos-related dynamics are of critical offensive relevance. First, Constrained Delegation allows a service to request a Service Ticket on behalf of any user. This abuse primitive is captured by our data extraction pipeline as an AllowedToDelegate edge, enabling the highly stealthy privilege escalation chains that our model learns to navigate. Second, while Kerberos secures standard authentication, ultimate domain compromise often bypasses ticket exchanges entirely. For instance, the DCSync attack leverages GetChanges and GetChangesAll replication rights to extract NTLM hashes directly from the Domain Controller without ever interacting with the Kerberos KDC.

NT LAN Manager (NTLM)

NT LAN Manager (NTLM) is a legacy suite of Microsoft security protocols that remains widely enabled in Active Directory environments, primarily to ensure backwards compatibility with older clients or poorly configured applications. Unlike the ticket-brokered architecture of Kerberos, NTLM utilizes a direct challenge-response mechanism. When a client attempts to authenticate to a server, the server issues a random challenge.

The client then encrypts this challenge using the NTLM hash of the user's password and returns it to prove their identity. Because NTLM lacks mutual authentication and transmits cryptographic material in a predictable challenge-response format, it is inherently weaker than Kerberos. Offensively, NTLM is frequently targeted for exploitation through hash extraction (Pass-the-Hash) and network interception (NTLM Relay attacks).

1.1.3 Trust Relationships

In large enterprise environments, organizations frequently span multiple domains or even entirely separate Active Directory forests. To allow users in one domain to access resources located in another, Active Directory employs a mechanism known as Trust Relationships [2]. A trust acts as a secure cryptographic bridge between two boundaries. These relationships can be configured as one-way, where Domain A trusts Domain B (but not vice versa), or two-way, where both domains trust each other equally. Furthermore, trusts can be transitive—meaning the trust flows through a chain of domains—or non-transitive, restricting the relationship strictly to the two interacting domains. As illustrated in the diagram below, these trust links can occur between child and parent domains within the same forest, or across entirely different corporate forests.

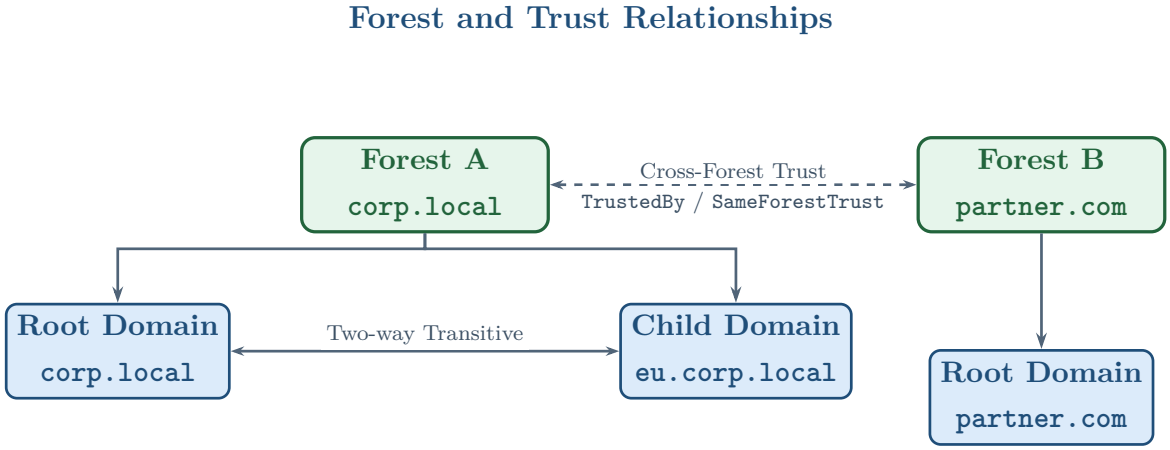

Figure 1.3: Active Directory Trust relationships.

While trust relationships are essential for administrative flexibility and resource sharing, they introduce complex attack vectors when misconfigured or overly permissive [4]. Adversaries frequently abuse bidirectional transitive trusts to move laterally across an entire forest boundary. For instance, if an attacker compromises a loosely secured child domain, they can forge an inter-realm ticket or abuse the SID History attribute to escalate privileges to the parent root domain, effectively compromising the entire forest. Similarly, poorly audited cross-forest trusts can inadvertently grant external users excessive permissions. This allows an attacker who compromises a partner organization's network to pivot seamlessly into the primary corporate environment.

Consequently, mapping and analyzing these trust links is a critical component of assessing an organization's true security perimeter.

1.2 Privilege Escalation in AD Environments

Offensive ACL Abuse and Structural Vulnerabilities

Access Control Lists (ACLs) constitute the granular security foundation of Microsoft Active Directory. They define the precise permissions—formally known as Access Control Entrys (ACEs)—that security principals (such as users, groups, or computers) hold over other directory objects. Historically, offensive security methodologies and defensive postures were heavily skewed toward network-level vulnerabilities, missing software patches, and memory corruption exploits. However, as Endpoint Detection and Response (EDR) solutions and patch management lifecycles have matured, modern offensive tradecraft has fundamentally pivoted. Adversaries now recognize that abusing misconfigured ACLs is often the most reliable, deterministic, and critical path for privilege escalation within an enterprise environment. [5]

Unlike traditional malware or binary exploitation, ACL abuse "lives off the land" by leveraging the intended, built-in features of the Active Directory architecture. Because these misconfigurations allow attackers to subvert the security model seamlessly—often without triggering traditional signature-based alerts—mapping these specific permissions has become the primary objective of post-compromise reconnaissance. Tools such as BloodHound have operationalized this mapping by modeling the directory as a graph, where nodes represent identity principals and edges represent the privileges they hold over one another. Consequently, these structural ACL relationships form the foundational dataset that the `GNN-AD-Navigator` pipeline extracts. By ingesting this topological data, the model learns to navigate the directory's complex trust architecture, fundamentally relying on these extracted rights to surface viable, end-to-end attack paths.

Operational Security OPSEC and Defensive Telemetry

In modern enterprise environments, offensive actions do not occur in a vacuum. The concept of Operational Security (OPSEC) is paramount when evaluating the practical viability of any simulated attack path. Blue Teams actively monitor directory environments using sophisticated detection mechanisms, including Security Information and Event Management (SIEM) architectures (such as Splunk or Elastic Security) and specialized identity threat detectors like Microsoft Defender for Identity (MDI). These defensive platforms continuously ingest telemetry from Domain Controllers, hunting for specific Windows Event IDs, anomalous API calls, and deviations from baseline

administrative behavior.

Consequently, modern attack paths can no longer be evaluated based solely on their topological distance (i.e., the shortest path). They must be rigorously evaluated on their "loudness" or detection footprint. A highly disruptive attack, such as forcing a password reset on a targeted administrator, might represent the shortest topological path to Domain Admin. However, this action generates immediate, high-fidelity alerts (such as Event IDs 4723 or 4724) and actively disrupts the legitimate user's access. This disruption frequently triggers helpdesk tickets and rapid incident response procedures, thereby immediately burning the attacker's foothold.

Conversely, a "stealthy" attack—such as modifying a delegation attribute or quietly adding a cryptographic Shadow Credential—might require a longer, multi-hop sequence but blends seamlessly into routine administrative traffic, bypassing legacy monitoring rules entirely. When an adversary or an automated red-team framework enumerates the directory to identify abusive rights, it must carefully weigh the defensive telemetry generated by the specific command-line tools and network protocols required to execute the exploit. This fundamental OPSEC reality underscores the core hypothesis of the `GNN-AD-Navigator` architecture: by utilizing a Graph Neural Network (GNN) with heterogeneous attention mechanisms, the model is explicitly designed to recognize the operational cost of different edge types, inherently penalizing "loud," heavily monitored transitions when stealthier alternatives exist.

1.2.1 DCSync Attack

The DCSync attack is one of the most critical techniques used by adversaries to extract credentials from a domain without requiring direct code execution on a Domain Controller [6]. Active Directory relies on the Directory Replication Service Remote Protocol (MS-DRSR) to synchronize data across multiple Domain Controllers. To abuse this, an attacker only needs an account that holds specific replication permissions—specifically, GetChanges (Replicating Directory Changes) and GetChangesAll (Replicating Directory Changes All) at the domain root level.

Once these permissions are acquired, the attacker's machine can masquerade as a legitimate Domain Controller and issue a replication request. The target DC will respond by transmitting the requested account credentials, including the highly sensitive NTLM hash of the `krbtgt` account. Securing these replication rights is often the terminal objective in offensive pathfinding operations, as it allows for the immediate creation of Golden Tickets and total forest compromise.

1.2.2 ADCS Attack Paths

Active Directory Certificate Services (ADCS) is Microsoft's native Public Key Infrastructure (PKI) implementation. While it provides essential cryptographic functions, misconfigured certificate templates introduce devastating escalation vectors [7]. These vulnerabilities, famously categorized into escalating scenarios (ESC1 through ESC15), often allow an unprivileged user to request a certificate on behalf of a highly privileged principal, such as a Domain Admin.

By leveraging the PKINIT protocol, the attacker can use the fraudulently obtained certificate to request a Kerberos Ticket Granting Ticket (TGT) for the impersonated admin, instantly seizing control of the domain. The inclusion of ADCS nodes and edges (such as Enroll and ManageCA) drastically expands the complexity of the Active Directory attack graph, presenting massive new navigational challenges for automated pathfinding tools.

1.2.3 SID History Abuse and RBCD

Beyond standard access controls, advanced adversaries frequently abuse inherent Active Directory features designed for compatibility and migration. The `sIDHistory` attribute, originally intended to preserve a user's access rights when migrating between domains, can be weaponized. If an attacker compromises a domain, they can inject the Security Identifier (SID) of a highly privileged group (such as Enterprise Admins) into an account's `sIDHistory`. This grants the attacker persistent, cross-domain administrative access while bypassing standard group membership audits.

Similarly, Resource-Based Constrained Delegation (RBCD) shifts the control of delegation from the domain administrator to the target resource itself [8]. If an attacker acquires GenericWrite privileges over a target computer object, they can modify its `msDS-AllowedToActOnBehalfOfOtherIdentity` attribute. This modification allows the attacker to authorize a computer they already control to impersonate any user on the compromised target, effectively granting them full administrative access to the machine. Because RBCD relies on targeted attribute modification rather than global delegation rights, it represents a stealthy and highly effective edge in modern attack graphs.

1.3 Existing Tools and Their Limitations

1.3.1 BloodHound and SharpHound

BloodHound has become the de facto industry standard for visualizing and analyzing Active Directory security structures [9]. Operating on a Neo4j graph database backend, BloodHound ingests data collected by its C# ingestor, SharpHound, to map relationships

between users, computers, and groups. By representing AD components as nodes and privileges as edges, BloodHound allows security professionals to visualize complex attack paths that would be impossible to identify manually.

However, BloodHound's primary limitation lies in its routing logic. The platform relies heavily on Dijkstra's algorithm to calculate the shortest path between an attacker's starting node and their target. In this deterministic model, every edge is treated with an equal, unweighted cost of one. BloodHound cannot natively distinguish between a highly detectable, noisy action (such as forcing a password reset on an active administrator) and a highly stealthy vector (such as abusing an AllowedToDelegate trust). Because it strictly minimizes hop count, the tool frequently recommends operationally unviable or easily detectable attack paths, leaving a critical gap for context-aware, machine learning-driven routing solutions.

1.3.2 `Certipy`

SharpHound historically lacked deep inspection capabilities for certificate infrastructure. To address this, `Certipy` was developed as a Python-based equivalent specifically for ADCS, designed for cross-platform and remote execution. Created by Oliver Lyak, Certipy is explicitly defined as an "offensive tool for enumerating and abusing Active Directory Certificate Services" [10]. Developed primarily for UNIX-like environments, this tool leverages remote queries to extract certificate templates, Certificate Authorities, and complex PKI permissions. It is an essential component for attackers operating from external infrastructure or non-Windows pivot machines, and it serves as the primary ADCS data collection engine for the pipeline developed in this thesis.

1.3.3 `bloodhound-python`

While SharpHound is designed to be executed directly from a compromised Windows host, `bloodhound-python` serves as its Python-based equivalent, designed for cross-platform and remote execution [11]. Developed primarily for UNIX-like environments, this tool leverages the `impacket` library to query LDAP and RPC remotely. It is an essential component for attackers operating from external infrastructure or non-Windows pivot machines, and it serves as the primary data collection engine for the pipeline developed in this thesis.

However, `bloodhound-python` suffers from severe functional limitations compared to its C# counterpart. Most critically, it lacks native support for enumerating ADCS templates. To bridge this gap, operators are forced to rely on external tooling such as `Certipy`. This introduces a fragile dependency chain: because Certipy officially retired BloodHound integration in its v5.0.0 release, operators must intentionally downgrade and maintain legacy versions (e.g., v4.3.0) strictly for dataset compatibility [12].

Furthermore, `bloodhound-python` exhibits blind spots in complex enterprise architectures, notably struggling to accurately map specific cross-domain trust accounts. Because its default implementation relies on static and often lagging extraction logic, the ingestor will blindly bypass novel misconfigurations or unmapped edge types unless its underlying parsing code is manually updated. This effectively blinds any downstream pathfinding algorithms to those specific routes, cementing the pipeline's extraction phase as a critical bottleneck for the overall model.

1.3.4 Other Automated Attack Path Tools

The success of BloodHound has inspired several supplementary tools aimed at automating attack path execution and reporting. Tools such as PlumHound and GoFetch interact directly with BloodHound's Neo4j database to generate human-readable audit reports or to automatically execute the identified exploit chains [13]. Similarly, auditing software like PingCastle provides rapid, rule-based assessments of Active Directory risk scores by flagging known misconfigurations [14].

While these tools excel at operationalizing graph data and generating compliance reports, they fundamentally inherit the baseline limitations of the underlying Neo4j Cypher queries. They do not generate novel paths or apply dynamic, environment-specific heuristics to evaluate edge costs. The offensive security industry currently lacks a widely adopted tool capable of applying heterogeneous attention mechanisms to weigh the semantic value of AD relationships. This thesis addresses this exact void by substituting deterministic shortest-path algorithms with a GNN policy capable of learning stealth and operational viability.

1.4 Graph Neural Networks

1.4.1 Message Passing and Neighbourhood Aggregation

Standard deep learning models are generally designed for structured data, such as grids of pixels or sequences of text. However, Active Directory (AD) environments are highly interconnected and asymmetrical, making them best represented as graphs. Graph Neural Networks (GNNs) address this by learning directly from the structural relationships between entities in the network.

Let an Active Directory environment be defined as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ represents the set of nodes (users, groups, computers) and $\mathcal{E}$ represents the set of edges (permissions, trusts, active sessions). The primary objective of a GNN is to encode each node into a continuous numerical vector, a **node embedding**. The initial state of a node $v \in \mathcal{V}$ is denoted by its starting feature vector, $\mathbf{h}_v^{(0)} = \mathbf{x}_v$.

To capture the topology of the network, GNNs rely on a mechanism called *message*

passing. During this process, each node gathers information from its direct neighbours to update its own state. Mathematically, the operation at the k-th **hidden layer** consists of an aggregation step and an update step. The message collected from the neighbourhood $\mathcal{N}(v)$ is defined as:

$$\mathbf{m}_{\mathcal{N}(v)}^{(k)} = \text{AGGREGATE}^{(k)}\left(\left\{\mathbf{h}_u^{(k-1)} : u \in \mathcal{N}(v)\right\}\right)$$

where AGGREGATE is a mathematical function such as the sum or mean. After aggregating this neighbourhood data, the node updates its own embedding. It combines its previous state with the newly collected message using a learned weight matrix $\mathbf{W}^{(k)}$ and a non-linear activation function σ:

$$\mathbf{h}_v^{(k)} = \sigma\left(\mathbf{W}^{(k)} \cdot \left(\mathbf{h}_v^{(k-1)} \parallel \mathbf{m}_{\mathcal{N}(v)}^{(k)}\right)\right)$$

The number of hidden layers, K, determines the receptive field of the model—meaning how far a node can "see" into the graph. For example, in a network with $K = 3$ layers, the final embedding $\mathbf{h}_v^{(K)}$ captures structural information from nodes up to three hops away.

Once the node embeddings are finalized, they are used to make routing decisions during an attack path simulation. This is managed by the **policy head**. Since the number of outbound connections varies greatly across AD nodes, the policy head evaluates each potential neighbour individually. It concatenates the current node's embedding $\mathbf{h}_v^{(K)}$, the candidate neighbour's embedding $\mathbf{h}_u^{(K)}$, and the target node's embedding $\mathbf{h}_{\text{target}}^{(K)}$, passing them through a Multi-Layer Perceptron (MLP) to generate a raw transition score:

$$e_{v,u} = \text{MLP}\left(\mathbf{h}_v^{(K)} \parallel \mathbf{h}_u^{(K)} \parallel \mathbf{h}_{\text{target}}^{(K)}\right)$$

These raw scores are then normalized across all valid neighbours using a Softmax function. This produces a valid probability distribution π that guides the pathfinding algorithm toward the most promising next step:

$$\pi(u|v) = \frac{\exp(e_{v,u})}{\sum_{w \in \mathcal{N}(v)} \exp(e_{v,w})}$$

The Impact of Graph Size and Density

While GNNs are highly effective, their performance is closely tied to the graph's size and connectivity. In large Active Directory networks, dense configurations—such as heavily nested group memberships—can cause *neighbourhood explosion*. This drastically increases the computational overhead during the message passing phase.

Additionally, increasing the number of hidden layers to expand the receptive field can

trigger *over-smoothing*. In this scenario, repeated aggregation causes the embeddings of structurally distinct nodes (e.g., a standard workstation and a Domain Controller) to blend together and lose their unique characteristics. Therefore, balancing the number of layers is crucial to maintaining the model's accuracy.

Finally, standard GNN architectures assume that all nodes and edges share the same properties. To properly model the diverse object types and distinct permission structures found in Active Directory, this foundation must be extended into a Heterogeneous Graph Transformer (HGT).

1.4.2 Heterogeneous Graph Transformer (HGT)

While a standard Graph Convolutional Network (GCN) aggregates neighbourhood features uniformly, Active Directory graphs are fundamentally heterogeneous. A MemberOf relationship carries drastically different operational semantics and stealth implications than an AllowedToDelegate or WriteDACL edge. The critical need for context-aware routing is currently driving rapid innovation in the offensive security space; for instance, recent 2025 research applying Physics-Informed GNNs to Active Directory graphs achieved an F1 score of 0.9308 when predicting paths across 16 distinct offensive edge types [15].

To capture this required semantic diversity, this research employs the HGT architecture proposed by Hu et al. [16]. HGT introduces a type-specific attention mechanism. Instead of sharing weights globally, it computes attention based on the specific meta-relation triplet $\tau = (\tau(s), \tau(e), \tau(t))$, representing the source node type, edge type, and target node type. The mutual attention score between a source node s and a target node t is calculated by projecting their embeddings into distinct Query and Key spaces via type-specific linear projections:

$$\text{Attention}(s,t) = \underset{\forall s \in \mathcal{N}(t)}{\text{Softmax}} \left(\Big\|_{i \in [1,h]} \frac{\mathbf{K}_s^{(i)} \mathbf{W}_{\tau(e)}^{(i)} \mathbf{Q}_t^{(i)^T}}{\sqrt{d/h}} \right)$$

This formulation allows the network to dynamically learn which specific attack vectors are most relevant when navigating between different entity types. Regarding the specific architecture implemented in this pipeline, the model utilizes an embedding dimension of 64, 4 attention heads, and is constrained to only 2 hidden layers. These hyperparameters act as a deliberate regularization strategy. The primary deciding factors were the physical size of the GOAD dataset during training, and the strict need to prevent the severe over-smoothing that occurs when aggregating features across dense Active Directory topologies.

1.4.3 PyTorch Geometric and `HeteroData`

To implement these mathematical frameworks, this project leverages PyTorch Geometric (PyG), a specialized extension library for deep learning on irregularly structured data **fey2019pyg**. Within PyG, the AD graph is constructed using the `HeteroData` format.

Traditional graph libraries store all connections in a single adjacency matrix, fundamentally stripping away semantic meaning. In contrast, the `HeteroData` architecture partitions the graph into distinct dictionaries based on meta-relations. Each relationship type—such as `(user, memberof, group)`—maintains its own isolated `edge_index` tensor of shape $[2, |\mathcal{E}_\tau|]$.

This isolated tensor structure is uniquely suited to the modern trajectory of offensive security. The industry is rapidly moving beyond isolated Windows domains; notably, the 2025 release of BloodHound Community Edition v8 introduced "OpenGraph," which expands pathfinding to include hybrid cloud identities, Entra ID, and third-party platforms like GitHub [17]. A homogeneous adjacency matrix would collapse under the introduction of these vastly different node and edge types. `HeteroData` natively supports this expansion by simply allowing the injection of new relationship dictionaries without restructuring the existing graph.

Furthermore, decoupling the graph structure from the ingestion logic provides massive pipeline resilience. As legacy data collection scripts face deprecation—such as Kali Linux 2025 enforcing strict Python 3.13 environments that break older Neo4j drivers [18]—our PyG-based pipeline remains agnostic. By storing the graph purely as isolated PyTorch tensors rather than relying on an active database connection, the GNN can cleanly mask out irrelevant relationships and accommodate future data collection tools with minimal friction.

1.5 Summary and Positioning

The application of deep learning to cybersecurity has historically been constrained by the rigid, Euclidean data structures required by traditional neural networks. However, enterprise networks are inherently non-Euclidean; they are highly complex webs of varying entities (users, computers, groups, policies) and heterogeneous relationships (authentication sessions, explicit permissions, cryptographic trusts). Graph Neural Networks represent the perfect theoretical fit for this domain, as they are natively capable of ingesting and learning from this structural and semantic diversity.

Despite this architectural alignment, the majority of applied GNN research in cybersecurity has been heavily skewed toward defensive anomaly detection. Models such as HetGLM [19] have demonstrated the efficacy of utilizing heterogeneous graphs to spot irregular lateral movement and anomalous authentication links. While highly valuable for Security Operations Centers (SOCs), these approaches model attacker behavior

retrospectively. They attempt to catch active intrusions probabilistically rather than mapping structural vulnerabilities proactively.

More recently, literature has begun to explore GNNs for proactive attack path prediction, notably through Physics-Informed Graph Neural Networks [15]. However, these approaches frequently suffer from a purely mathematical, theory-driven bias. Because these models are often developed outside the context of applied offensive security, they rely heavily on auto-generated, synthetic datasets that fail to replicate the messy idiosyncrasies of real-world Active Directory deployments. Furthermore, rather than learning the true operational cost of a specific exploit—such as weighing the stealth of an AllowedToDelegate abuse against the high detectability of a ForceChangePassword action—these models often devolve into simply replicating BloodHound's shortest-path Dijkstra logic across larger matrices.

This thesis directly addresses that gap. Rather than approaching the directory graph as a pure mathematical traversal problem on synthetic data, this research positions the GNN as an offensive agent. By leveraging the Heterogeneous Graph Transformer architecture and training it against operationally accurate environments, the pipeline developed in this work learns a nuanced offensive navigation policy. It does not simply seek the mathematically shortest path; it learns the most operationally viable route, effectively bridging the gap between academic graph theory and practical penetration testing tradecraft.

Chapter 2

System Architecture and Design

2.1 System Overview

The GNN-AD-Navigator is designed to process and analyze any Active Directory (AD) environment. It maps raw BloodHound scans into traversable PyG nodes and edges, allowing the model to navigate and score potential attack transitions. Furthermore, given a BloodHound-compatible Certipy scan, the tool is capable of stitching missing ADCS templates back into a unified PyG graph.

To achieve this, the pipeline utilizes a series of targeted scripts to construct the final `heterodata.pt` tensor object. These scripts handle the merging of dissected scans (for example, scans representing different domains or collected from different privilege levels), the injection of ADCS templates, and the cleanup of dangling nodes and edges. A validation script then ensures data integrity while generating clear, readable logs.

Operational Modes and Design Decisions

To accommodate different stages of a red team engagement, the pipeline was built with three primary operational modes, managed via the `launch.sh` wrapper:

1. **Data Preparation Only:** Parses the raw JSON files, stitches the graphs, and builds the tensors without querying the AI.
2. **Preparation and Query:** Builds the tensors and immediately launches the inference script from a specified start node to a target node.
3. **Inference Only:** Once the `heterodata.pt` object is created the user can skip the data preparation phase and run the pathfinding algorithm directly.

A principal design decision maintained throughout this pipeline is the enforcement of the BloodHound-standard principal-to-target edge directionality. This ensures that the simulated attacker can only traverse edges where they mathematically hold the source privilege.

Path Discovery and Auditing

Once the graph accurately mirrors the target AD environment, the inference phase begins. The `beam_search` algorithm queries the model's policy head to navigate the graph, discovering viable attack paths and ranking them based on the network's confidence scores. Finally, the tool runs an operational audit against static rules, evaluating the output and reminding the user to carefully review each path before proceeding with active exploitation.

2.2 Data Collection and Preprocessing

The launch.sh script serves as the primary execution pipeline for the GNN-AD-Navigator, orchestrating both the data preparation and model inference phases. During the preparation stage, it categorizes raw JSON outputs from BloodHound and Certipy, cleans the data, and merges it into a unified Active Directory forest graph. It then translates this combined graph into tensor formats `heterodata.pt` required by PyTorch. Finally, the script can execute an inference query using either a GCN or HGT model to calculate potential attack paths between a user-defined starting node and target node.

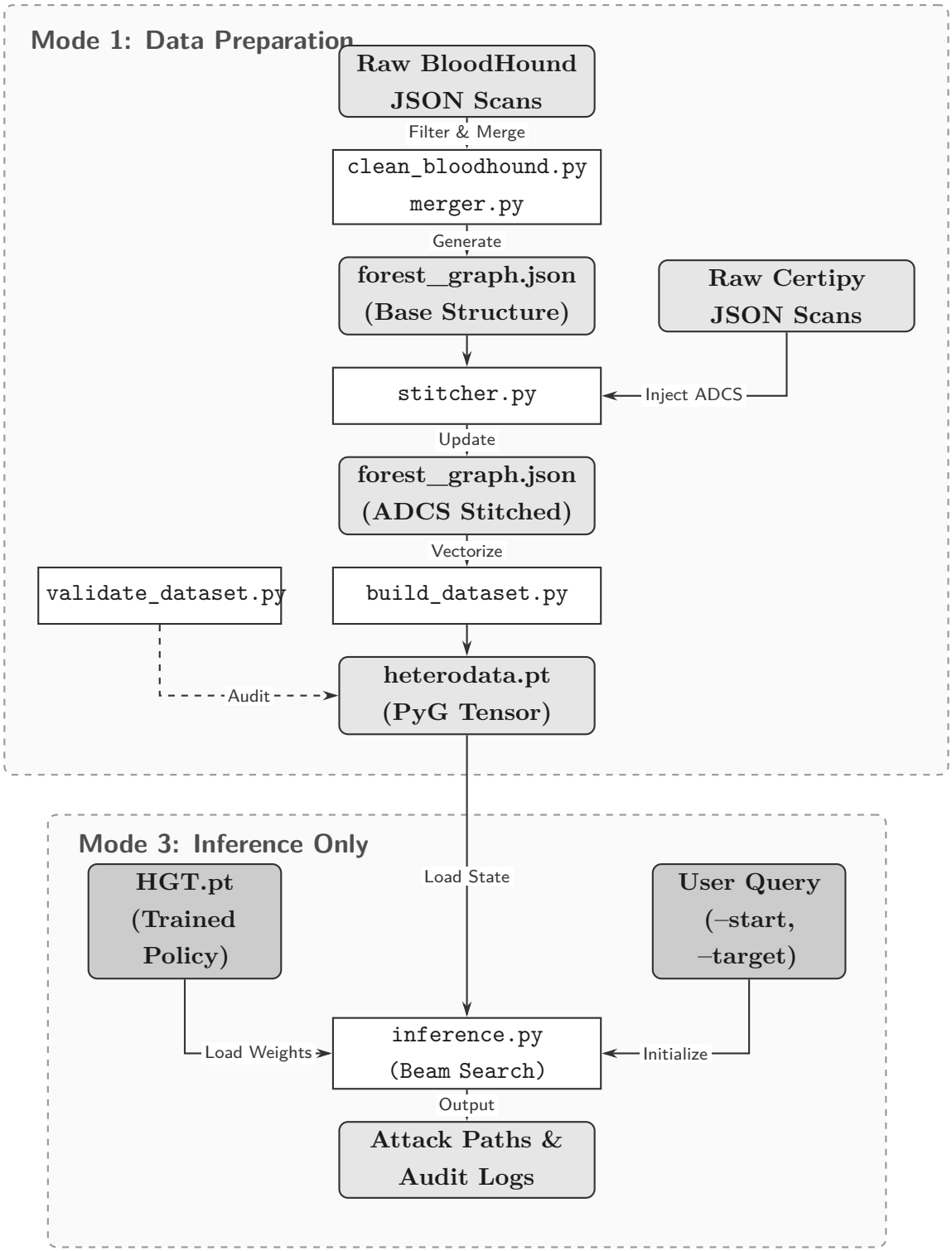

Figure 2.1: Data Pipeline flow.

2.2.1 `clean_bloodhound.py`: Filtering and Multi-Domain Accumulation

The initial stage of the data preparation pipeline is handled by the `clean_bloodhound.py` script, which is responsible for sanitizing raw BloodHound JSON outputs. To support large-scale enterprise environments, the script implements a recursive, multi-domain ac-

cumulation mechanism. This ensures that when input data contains scans from multiple domains or is split across various files, no topological data is silently overwritten. The script accomplishes this by deduplicating entities based on their `ObjectIdentifier` (SID), selectively retaining the "richer" JSON record if duplicate scans of the same domain occur.

Crucially, this component acts as a noise-reduction filter to optimize the GNN's computational overhead. It builds an initial edge index across all entities to determine topological connectedness. Using this index, the script systematically drops irrelevant nodes. For example, `Computer` nodes matching standard workstation naming conventions (e.g., `WKS-0`) are pruned if they lack dangerous ACL privileges, unconstrained delegation rights, and possess an edge degree of one or less. Similarly, disabled `User` nodes are purged from the dataset unless they retain active sessions or explicit ACL edges, while empty `Group` objects with no members or outbound privileges are entirely discarded. Nodes representing Domains, Group Policy Objects (GPOs), and Organisational Units (OUs) are preserved as-is to maintain structural hierarchy.

2.2.2 `merger.py`: Normalisation and Merging

Following the initial filtering and ADCS data stitching, the pipeline executes `merger.py`. This script functions as a generic forest merger, capable of accepting heterogeneous inputs such as directories of cleaned JSON files or previously stitched domain graphs and consolidating them into a unified `forest_graph.json` structure. Positioned directly before the tensor construction phase, this module is strictly responsible for cryptographic and structural alignment.

To ensure flawless tensor mapping in PyTorch Geometric, the script enforces a strict normalization protocol. All `ObjectIdentifier` and `PrincipalSID` values, including cross-domain `TargetDomainSid` references, are strictly cast to uppercase, while key categorical properties (such as `name`, `domain`, and `RightName`) are normalized to lowercase.

When merging overlapping datasets, the script resolves conflicts through a deterministic scoring heuristic. The `score_obj` function evaluates node richness by awarding points for populated properties, active ACL rules, and group members, while applying a heavy multiplier to explicit cross-domain `Trusts` . The conflicting entity with the highest topological score is retained. Finally, the script executes a validation routine that inventories all discovered trust relationships, aggregates edge type frequencies, and calculates a "dangling-reference rate" to quantify relational integrity before the data is vectorized into the GNN.

2.2.3 `stitcher.py`: ADCS Injection and Domain Detection

Because Certipy operates independently of BloodHound, its raw JSON output lacks the native topological bindings required to merge seamlessly into an existing AD graph. To bridge this gap, the pipeline employs `stitcher.py`, an intermediate augmentation script positioned directly between the filtering and merging phases. Before the script is even invoked, the pipeline's overarching wrapper automatically parses the Certipy output to dynamically detect the target domain. By analyzing the Certificate Subject or Distinguished Name embedded within the scanned certificates, the wrapper extracts the domain context and passes it to the stitcher without requiring manual operator input.

Once invoked, the script injects two novel node categories into the graph: Certificate Authority (CA) and `certtemplates`. Because Certipy does not natively retrieve SIDs for these entities, the script generates deterministic, synthetic Ghost SIDs (e.g., `S-1-5-21-GHOST-...`) by computing an MD5 hash of the template name and the target domain. To connect these isolated nodes to the broader network, the script builds a resolution dictionary that maps lowercase principal names to their existing `ObjectIdentifier` within the forest. It then translates complex ADCS relationships into standard BloodHound `Aces` entries, injecting edges such as Enroll, PublishedTo, and WriteDACL. Critically, it also injects an HttpEnroll edge to model ESC8 vulnerabilities, and a TrustedForNTAuth edge to link the CA back to the domain root, preventing the routing algorithm from dead-ending.

Beyond explicit ADCS integration, `stitcher.py` acts as the engine for synthetic Advanced Persistent Threat (APT) vector generation. It proactively scans the forest for logically vulnerable delegation configurations. If a source entity possesses `unconstraineddelegation` rights, the script dynamically generates a coercetgt edge to mathematically represent NT LAN Manager (NTLM) coercion attacks (e.g., PetitPotam). To preserve topological integrity, this script strictly validates that the target is an unprotected Domain Controller computer account, strictly preventing the GNN from calculating illogical attack paths through abstract entities like OUs or standard user accounts.

2.2.4 Training Environment: GOAD

The Game of Active Directory (GOAD), developed by Mayfly, serves as the primary training and diagnostic environment for this research. GOAD is a highly complex, multi-domain, and multi-forest vulnerable lab designed specifically to simulate advanced, real-world enterprise architectures. The environment is composed of five virtualized Windows Server machines running Windows Server 2016 and 2019 distributed across two distinct forests. The primary forest contains a parent domain `sevenkingdoms.local` and a child

domain `north.sevenkingdoms.local`, simulating a typical corporate hierarchy with bidirectional transitive trusts. A secondary, external forest `essos.local` is connected via a forest-to-forest trust with SID history enabled. This sophisticated topological layout provides the heterogeneous graph structure required to train the Graph Neural Network on cross-domain lateral movement and complex trust exploitations.

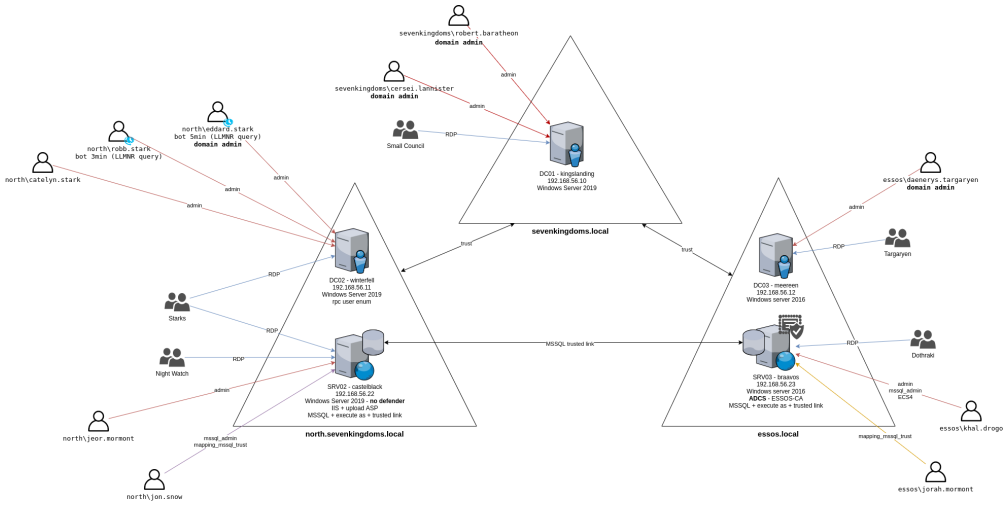

Figure 2.2: The logical architecture and domain topology of the GOADv2 environment. *Source: Mayfly (2022).*

Beyond basic network topography, GOAD is engineered with deeply embedded, configuration-level vulnerabilities that test the limits of automated attack path discovery. The environment features multi-step ACL kill-chains, deliberately exposed ADCS configured to be vulnerable to ESC1 through ESC3 and ESC8 relay attacks, and various forms of delegation (Unconstrained, Constrained, and Resource-Based Constrained Delegation). By exposing the Navigation Model to this dense concentration of intersecting vulnerabilities, the network learns to evaluate competing escalation routes—such as choosing between a force change password against an IIS server or a stealthy `WriteDACL` abuse—making GOAD an ideal foundational dataset for imitation learning in offensive security. The attack paths goad provides are to be presented in a learnable and readable training format for the model. Each attack path is deconstructed into transitions, `current_node`, `next_hope`, `credentials_used`, `total`, `path_weight`, `train_on_step`, `notes` and the `step_cost`.

2.2.5 Feature Engineering

To a GNN model learning from a bloodhound scan, attack paths are of different importance, some are totaly invisible while others are a gold mine for what the model needs to replicate. Paths that rely on unpatched vulnerabilities or outdated systems, for instance, leave no trace in the BloodHound graph and are marked with a `path_weight` of 0. Paths that are mostely visible with only two or less transitions that do not

qualify have the latter marked through `train_on_step` set to zero. And at last, for an imitation learning model, given the choice between two valide transitions, the model choses based on training data, prefering the edge type it has seen most during training. To address this We are to more or less guide the model to gravitate towards the edge attack with less impact on the environment, the one with less noise and impact. To reach the desired effect each valid transition is scored. This score is reflected directly in the loss function through the `step_cost` feature.

Table 2.1: Step cost assignment by attack category.

Attack Category	Cost Range	Rationale
Topological / Logical	0.1 – 0.2	Operations such as adding a member to a group or enrolling in a certificate template. These transitions are structurally clean and rarely trigger alerts unless heavy auditing is configured.
Credential Abuse	0.3 – 0.4	Using a recovered password or a forged TGT. Low noise overall, but detectable by honeytokens or behavioural analysis tools.
Active Enumeration	0.5	Kerberoasting or intensive LDAP queries. Intermediate noise level; commonly detected by modern SIEMs monitoring ticket requests or query volume anomalies.
Memory / Disk Dumping	0.6 – 0.7	Dumping the SAM database or LSASS process memory. High detection risk; endpoint protection tools such as Microsoft Defender and CrowdStrike flag these operations near-immediately.
Network Exploitation	0.8 – 0.9	Active coercion and relay attacks such as PetitPotam or NTLM Relaying, and brute-force techniques such as password spraying. Very noisy; triggers IDS/IPS rules and account lockout policies.

The `step_cost` value assigned to each transition feeds directly into the loss function through the effective weight computation:

The feature engineering phase introduced three mechanisms: filters to eliminate CVE-dependent transitions invisible to the graph collector; path weights reflecting a composite visibility score that prioritises transitions where the graph structure itself is the primary escalation vector; and a `step_cost` feature guiding the model toward stealthier, less impactful edges when a choice presents itself. This constitutes a simplified reward shaping approach, consistent with standard practices in the imitation learning literature.

2.3 Navigation Model

2.3.1 HGT Encoder

To effectively process the highly asymmetrical and heterogeneous structure of AD—which contains distinct entity types (e.g., users, computers, groups, domains) alongside a complex vocabulary of privilege edges—the architecture utilises an HGT encoder [20]. Implemented via the PyG library, the encoder is specifically designed to handle the native topological variations without requiring manual modification of the graph, preserving the distinct semantic meaning of each node and edge type as established in the foundational HGT literature.

The encoding phase begins by projecting the disparate input features of each node type into a unified dimensional space. This is achieved by passing the raw features through type-specific linear projection networks, mapping every object into a standardised 64-dimensional hidden vector. Once projected, the architecture applies a series of HGT convolutional layers utilising multi-head attention mechanisms to propagate context across the network. Specifically, the model employs four attention heads across two sequential message-passing layers. This bounded depth prevents the over-smoothing of node features while still allowing the network to aggregate vital context from a node's local topology.

Through this mechanism, the encoder intrinsically learns the semantic value of various edge types—for example, mathematically distinguishing between a standard `GenericWrite` and a high-privilege `MemberOf` relationship. The final output of the encoder is a dense, concatenated embedding matrix where every object in the AD network is represented as a sophisticated 64-dimensional vector that encapsulates its structural privilege and potential attack utility.

2.3.2 Policy Head

Following the extraction of topological features, the navigation model must evaluate and rank potential escalation paths. This decision-making phase is governed by the Policy Head. During traversal, the inference algorithm isolates the attacker's current

node and identifies all topologically valid outbound neighbours.

To score these candidate transitions, the Policy Head leverages the embeddings generated by the HGT encoder. Rather than computing a rigid concatenation of the current state and a fixed target state, the model evaluates the transition via the Hadamard product (element-wise multiplication) between the current node's expanded embedding and each candidate neighbour's embedding. This element-wise interaction matrix is then processed through a Multi-Layer Perceptron (MLP) consisting of consecutive linear transformations, Rectified Linear Unit (ReLU) activations, and dropout layers.

The MLP ultimately compresses this latent relationship into a single scalar confidence score for each candidate. By relying on the relative, multiplicative interaction between node embeddings, the policy head acts as a generalised privilege evaluator. It steers the algorithmic traversal toward high-value nodes based strictly on the structural context learned during the encoding phase, completely decoupling the pathfinding logic from hardcoded heuristic rules.

2.3.3 Loss Function and Step Cost

To train the policy head to navigate not just efficiently, but stealthily, the architecture employs a weighted cross-entropy loss function. The effective training weight for each transition is calculated as follows:

$$\text{eff\_weight} = \text{path\_weight} \times (1 - \text{step\_cost}) \tag{2.1}$$

where `path_weight` reflects the overall visibility and quality of the full attack path, and `step_cost` penalises individual transitions according to their noise category in Table 2.1. A high `step_cost` reduces the effective training weight of that transition, making the model less likely to replicate it when a lower-cost alternative exists in the training data. Conversely, topological transitions with a low `step_cost` receive a higher effective weight and are reinforced more strongly during training. The result is a model that, when faced with a choice between two structurally valid transitions, gravitates toward the one with less environmental impact — not because it was explicitly told to, but because the training signal made that choice more frequent and more rewarded.

This reward shaping mechanism fundamentally alters the pathfinding paradigm compared to traditional graph traversal algorithms. Standard shortest-path algorithms, such as Dijkstra's algorithm implemented in default BloodHound, traverse the network by treating all edges as having an equal weight of one. Consequently, they inherently optimise for the shortest structural distance, which frequently results in highly detectable, noisy exploit chains.

By integrating the `step_cost` directly into the loss function, the neural network is mathematically discouraged from selecting loud transitions during the imitation learning

phase. Instead of merely memorising the shortest routes from the training corpus, the policy head internalises an OPSEC-driven heuristic. During inference, this enables the model to discover non-linear, multi-hop Living of the Land (LotL) paths that exhibit a significantly lower cumulative detectability cost than routes generated by unweighted Dijkstra searches on the exact same topology. This capability to prioritise stealth over topological proximity forms the core justification for deploying deep learning in automated offensive navigation.

2.3.4 Beam Search Navigation

Algorithm

The core pathfinding engine during inference is a heuristic beam search algorithm designed to evaluate and navigate the topological structures extracted by the HGT encoder. At each transition step, the algorithm queries the policy head to score all valid outbound neighbours, converting the raw logits into a probability distribution via a Softmax function. Rather than selecting the single best transition, the beam search maintains a tracked list of the top k candidate paths (the beam width), expanding each and accumulating their transition probabilities logarithmically to prevent mathematical underflow.

Crucially, the algorithm includes a fallback mechanism for undocumented or Out Of Vocabulary (OOV) edge types. If the inference engine encounters a relationship that the neural network did not observe during training, it bypasses the policy head's probabilistic output and assigns a static baseline operational score of 0.05. This ensures the pipeline remains resilient and does not crash when deployed against evolving AD environments with novel schema additions.

Algorithm 1: Heuristic Beam Search for Attack Path Discovery

Input: Graph G, Start Node S, Target Node T, Beam Width k, Max Depth D
Output: Top k Attack Paths

```
beam ← [(score: 0.0, path: [S])]
completed ← []
for d ← 1 to D do
    candidates ← []
    foreach (score, path) ∈ beam do
        C ← path[−1]
        if C = T or HasDCSync(C, T) then
            completed.append((score, path))
            continue
        end
        neighbours ← GetValidNeighbours(C)
        probs ← PolicyHead(C, neighbours)
        foreach N, prob ∈ (neighbours, probs) do
            if IsOOVEdge(C, N) then
                prob ← 0.05
            end
            new_score ← score + log(prob)
            candidates.append((new_score, path + [N]))
        end
    end
    beam ← TopK(candidates, k)
    if length(completed) ≥ k then
        break
    end
end
return TopK(completed ∪ beam, k)
```

Terminal State:* *Directory Replication Service synchronisation attack (DCSync)

To optimise traversal and mimic the operational objectives of a professional red team, the inference engine employs a dynamic, state-aware termination condition rather than blindly searching for an exact target node match. At each step, the algorithm evaluates whether the current node possesses DCSync privileges over the target's domain. Specifically, it checks for the combination of `GetChanges`, `GetChangesAll`, and `GetChangesInFilteredSet` directory replication rights.

If a path reaches a node holding these privileges, the algorithm recognises global reachability and immediately halts, trimming any subsequent expansion . This assumes total domain compromise, acknowledging that an attacker with DCSync rights can organically forge the remaining transitions to reach the final objective without requiring further algorithmic guidance.

Cross-Environment Inference and Global Coordinate System

A critical engineering challenge during deployment is bridging the gap between the strictly heterogeneous data structures required by the neural network and the homogeneous, indexed lookups required by classical search algorithms.

During the initial ingestion and encoding phase, the graph strictly maintains its heterogeneity. The HGT processes the environment using isolated feature dictionaries and type-specific edge indices, ensuring the message passing mathematically preserves the distinct semantic nature of users, computers, and disparate AD permissions.

However, once the latent embeddings are extracted, tracking complex multidimensional tuples (e.g., `('computers', 12)`) during a rapid, multi-hop heuristic beam search introduces severe computational overhead. To resolve this, the inference module dynamically constructs a unified global coordinate system post-encoding.

The heterogeneous embeddings are concatenated into a singular, flattened tensor space, and all structural edges are mathematically shifted using pre-calculated type offsets. This creates a global adjacency mapping for the deterministic search algorithm. It allows the beam search to execute highly efficient integer-based traversals (e.g., jumping from global Node 15 to Node 102) while still querying the policy head with the rich, heterogeneously derived latent vectors. This dynamic ingestion and coordinate resolution allow the model to execute inferences on completely unseen, enterprise-scale forests without requiring any structural modifications to the base HGT.

2.3.5 Audit Module

Because deep learning models operate probabilistically, the pipeline includes a deterministic audit module to evaluate the semantic quality and operational viability of the generated attack paths. The module categorises outputs into three distinct reliability tiers:

- **Ok:** The path successfully reaches the explicit target node or terminates at a node possessing direct DCSync privileges over the target domain, signifying a complete operational success.
- **Warn:** The path identifies a critical escalation route but requires human improvisation to complete. For example, this occurs if the model achieves DCSync on a

foreign domain, necessitating a cross-forest SID History attack, or if the beam search terminates structurally with noisy edges such as `MemberOf` or `Contains` without reaching the final target.

- **Fail:** The path represents an operational dead-end. This is triggered if the beam search is unable to progress past the starting node or terminates at an unprivileged object lacking domain replication rights.

By pairing neural pathfinding with this strict heuristic audit, the tool ensures operators receive transparent, actionable intelligence alongside clear diagnostics when the algorithmic routing falls short.

2.4 Validation Module & Logging

To ensure operational transparency and deterministic execution, the architecture implements a centralised logging framework that captures both standard output and error streams across all pipeline stages. From the initial heterogeneous data ingestion to the final algorithmic traversal, every data mutation, structural shift, and inference decision is immutably recorded. By multiplexing the execution stream—providing operators with real-time diagnostic feedback while simultaneously committing telemetry to a persistent log—the system guarantees a comprehensive historical audit trail for subsequent evaluation or architectural debugging.

Prior to initiating the neural network or training phases, a dedicated validation module systematically audits the extracted graph tensors and the supervised training corpus against a strict set of expected baselines. It evaluates the mathematical health of the embeddings, confirms the preservation of critical operational pathways (such as ADCS escalation routes), and eliminates topological anomalies like self-loops. Crucially, it cross-references the expected outcomes—the documented target hops within the training examples—against the constructed graph to ensure all objective coordinates accurately map to valid boundaries within the heterogeneous tensors. By classifying structural deviations as either informational warnings or fatal execution blockers, the module guarantees that only mathematically sound and topologically viable data reaches the HGT encoder.

Chapter 3

Implementation

3.1 Technical Stack

The `GNN-AD-Navigator` pipeline is engineered entirely in Python, leveraging an ecosystem of specialised libraries for deep learning, graph processing, and interactive visualisation. The core neural network architecture and heterogeneous tensor operations are implemented using `PyTorch` and `PyTorch Geometric (PyG)`. For graph topology manipulation and deterministic traversal algorithms (such as the heuristic beam search), the system relies heavily on `NetworkX`.

To provide operators with an accessible and interactive frontend, the audit module and visualisations are built using `Streamlit`, coupled with `Pyvis` for dynamic, browser-based rendering of the discovered attack paths.

Model training was offloaded to cloud-based Kaggle compute instances to manage the heavy matrix multiplications required by the Heterogeneous Graph Transformer. Specifically, the training pipeline was pinned to dual NVIDIA Tesla T4 Graphical Processing Units (GPUs). This specific hardware selection was a deliberate engineering constraint; older architectures like the Tesla P100 (which utilise the `sm_60` compute capability) exhibit architectural incompatibilities with the compiled CUDA binaries of modern `PyTorch` releases.

To ensure strict reproducibility across environments, the core dependencies were version-pinned as detailed in Table 3.1.

Library / Framework	**Version**
Python	3.10.x
PyTorch	2.1.0+cu118
PyTorch Geometric (PyG)	2.4.0
NetworkX	3.1
Streamlit	1.28.0
Pyvis	0.3.2

Table 3.1: Core technical stack and version pinning for environment reproducibility.

3.2 Repository Structure

The project repository is organised into a highly modular, pipeline-driven architecture. This separation of concerns ensures that data ingestion, neural network training, and frontend visualisation remain strictly decoupled. The directory tree is structured as follows:

```
gnn-ad-navigator/
├── examples/           # Sample AD graphs and expected pipeline outputs
├── gui/                # Streamlit frontend (app.py, visualisation.py)
├── lib/                # Core modules and helper functions
├── models/             # Serialised PyTorch weights (.pt checkpoints)
├── notebooks/          # Jupyter/Kaggle notebooks for model training
├── scripts/            # Core pipeline execution scripts
│   ├── build_dataset.py
│   ├── clean_bloodhound.py
│   ├── inference.py
│   ├── merger.py
│   ├── prepare_training_examples.py
│   ├── stitcher.py
│   └── validate_dataset.py
├── thesis/             # LaTeX source code for the manuscript
├── .dockerfile         # Containerised deployment configuration
├── .gitignore          # Strict ignore rules for sensitive data
├── launch.sh           # Master bash script for pipeline execution
├── README.md           # Documentation and setup instructions
├── requirements.txt    # Python dependencies
└── setup.sh            # Environment initialisation script
```

A critical security consideration within the repository's configuration is the `.gitignore` file. Given the highly sensitive nature of Active Directory environments, the repository enforces strict exclusion paranoia. All `.json` scans, `Certipy` outputs, and compiled `.pt` tensors are explicitly ignored by version control to ensure that no real-world or client-specific infrastructure data is ever inadvertently committed to the public history, The only exception being the included test environment INLANEFREIGHT.local scans.

3.2.1 Script Responsibilities and Data Flow

To maintain strict modularity and ensure reproducibility across disparate environments, the execution of the data pipeline is fully decoupled from the underlying Python scripts.

The entire lifecycle—from raw data ingestion to algorithmic inference—is managed by a master shell orchestrator (`launch.sh`).

This orchestrator sequentially routes data through a series of specialised Python micro-scripts, handling standard output logging, error trapping, and dependency validation between stages. It bifurcates the system into two distinct operational modes: a *Data Preparation* phase, which compiles the heterogeneous tensors, and an *Inference* phase, which executes the heuristic beam search.

The precise data flow, detailing the structural transformation of information at each discrete stage of the pipeline, is outlined in Table 3.2.

Stage	Script	Primary Input	Output Artifact
1. Filter	`clean_bloodhound.py`	Raw BloodHound `.json`	Cleaned BloodHound `.json`
2. Merge	`merger.py`	Cleaned BloodHound `.json`	`forest_graph.json` (Base)
3. Stitch	`stitcher.py`	Base Graph + Certipy `.json`	`forest_graph.json` (ADCS Stitched)
4. Vectorise	`build_dataset.py`	Stitched `forest_graph.json`	`heterodata.pt` (PyG Tensor)
5. Audit	`validate_dataset.py`	`heterodata.pt`	Telemetry & Validation Logs
6. Inference	`inference.py`	`heterodata.pt` + User Query	Attack Paths & Audit Logs

Table 3.2: Sequential data flow and script responsibilities within the GNN-AD-Navigator pipeline.

By strictly isolating responsibilities, the pipeline ensures that if environmental collection tools (such as BloodHound or Certipy) alter their output schemas in the future, only the initial filtering and stitching scripts require modification, leaving the core mathematical vectorisation and inference engines completely untouched.

3.2.2 SharpHound Version Normalisation

A known architectural fragility within the data ingestion pipeline stems from the evolutionary nature of third-party enumeration tools. As collectors like SharpHound and `bloodhound-python` receive updates, their underlying `json` output schemas frequently change. Most notably, the nomenclature used to define specific privilege edges is prone to version drift.

Because the Heterogeneous Graph Transformer (HGT) and the underlying PyG framework require a strictly defined, static edge vocabulary during tensor initialisation, encountering an undocumented or altered edge string during ingestion will either crash the vectorisation process or cause the pipeline to silently drop critical topological data.

To mitigate this schema volatility, the data pipeline implements two explicit normalisation strategies prior to tensor construction:

- **Casing Normalisation:** Edge types frequently fluctuate between camel-case and upper-case representations across different tool versions (e.g., `WriteDACL`

versus `WriteDacl`). The `merger.py` script resolves this by enforcing aggressive, graph-wide lowercase normalisation on all relationship strings during the initial graph synthesis.

- **Semantic Aliasing:** Tool updates occasionally deprecate entire relationship names in favour of new terminology. For example, legacy trusts represented as `TrustedBy` must mathematically map to the modern `SameForestTrust` tensor to ensure the neural network recognises the transition. The `build_dataset.py` script implements a static aliasing dictionary to intercept and translate these known schema variations.

These interventions are documented as known environmental fragilities rather than software bugs. They underscore the necessity of the pipeline's modular design, ensuring that string-level normalisation is handled exclusively at the data preparation layer so the core mathematical vectorisation engine remains isolated from third-party schema drift.

3.3 Training

3.3.1 Training Data: GOAD

As established in the theoretical framework, the GOAD (Game of Active Directory) laboratory serves as the foundational, multi-domain environment for this research. To generate the supervised training corpus, a series of deterministic attack paths were manually executed within this environment and transcribed into a structured trace file.

While the raw trace logs contain nearly 50 discrete execution steps, the dataset was rigorously filtered to isolate purely topological transitions. Steps requiring out-of-band execution (such as dropping web shells or exploiting ghost steps) were pruned, resulting in a refined subset of approximately 33 learnable, structurally valid edges. These curated transitions were subsequently processed into 52 distinct training examples (comprising 26 constrained and 26 unconstrained permutations) to populate the PyTorch Geometric `HeteroData` tensor.

To ensure the policy head develops both precise routing capabilities and generalised escalation heuristics, these examples are split into a dual-mode training structure:

- **Constrained Examples:** The transition traces provide the model with both an explicit starting node and a predefined target destination. This teaches the network how to mathematically route an attack chain toward a specific coordinate within the graph.

- **Unconstrained Examples:** The traces provide a starting point, but the target is replaced with a `<null>` sentinel node. This simulates open-ended exploratory

escalation, training the model to intrinsically recognise universally high-value targets (such as Tier-0 groups or Certificate Authorities) even when an explicit operational objective is not provided.

The primary objective of these traces is to teach the HGT encoder the latent semantic hierarchy of native AD permissions. The training corpus heavily emphasises ACL exploitation chains (e.g., `GenericAll`, `WriteDacl`), nested group inheritance via `MemberOf`, and cross-domain trust transversals, consistently guiding the model toward the DCSync terminal state.

Crucially, this curated dataset establishes a strict coverage ceiling for the model's operational awareness. The training traces deliberately exclude transitions reliant on underlying host vulnerabilities, meaning the policy head remains blind to Common Vulnerabilities and Exposuress (CVEs) such as PrintNightmare or ZeroLogon. Furthermore, owing to the baseline scope of this specific corpus, advanced modern primitives—such as RBCD (Resource-Based Constrained Delegation), Kerberoasting, and Shadow Credentials—are absent from the learnable tensor transitions. This boundary is explicitly acknowledged: the neural network cannot reliably infer or prioritise attack vectors that fall completely outside its supervised training vocabulary.

3.3.2 Kaggle Training Notebook

The neural network training was executed via a structured Kaggle notebook environment to leverage dedicated hardware acceleration. The script handles the dynamic ingestion of the PyG `HeteroData` object, extracting the discrete arrays of node and edge types to dynamically initialise the dimensions of the HGT encoder. Following instantiation, the notebook executes the training loop over the curated constrained and unconstrained permutations, optimising the policy head via the custom weighted cross-entropy loss function.

A critical engineering implementation within this notebook resides in the model exporting mechanism. Standard PyTorch checkpointing typically only saves the raw mathematical weights (the `state_dict`). However, the training script injects a custom dictionary patch that explicitly bundles the graph's structural metadata—specifically the array of node types, edge types, the hidden dimension size, and the input feature dimension—directly alongside the exported weights.

3.3.3 Model Checkpoints

The culmination of the training pipeline is the generation of the serialised `HGT.pt` model checkpoint. The metadata patch bundled within this file is not merely a logging convenience; it is a strict mathematical necessity for conducting cross-environment inference.

Without the bundled metadata, executing inference in a novel AD environment introduces severe architectural fragility. If a target enterprise forest lacks a specific relationship type that was present in the training data (or introduces an entirely new, unseen edge), initialising the HGT dynamically based on the new target graph's schema will result in mismatched tensor dimensions. This causes either a hard compiler crash or, far more dangerously, a silent architectural misalignment where the trained neural weights are shifted and applied to the wrong privilege edges.

```
# Added metadata and num_features for cross-environment
   inference
torch.save({
    "epoch":        epoch,
    "model_state":  model.state_dict(),
    "loss":         loss,
    "architecture": "HGT",
    "hidden_dim":   HIDDEN_DIM,
    "num_features": feature_dim,
    "metadata":     metadata
}, CHECKPOINT_HGT)
```

Listing 3.1: Injecting topological metadata into the PyTorch checkpoint.

By loading the exact structural metadata bundled inside `HGT.pt`, the `inference.py` script forces the PyTorch engine to reconstruct the precise neural architecture present during training. The pipeline can then safely project the target environment's data into this static schema. Node types present in the model but missing in the target environment are safely padded with zero-tensors, and novel edges are bypassed using the beam search's out-of-vocabulary fallback logic. This mechanism ensures the model's structural integrity remains completely deterministic, regardless of the target environment's topological anomalies.

3.4 Graphical & Command Line Interface

Operational Context and Future Integrations

From an architectural standpoint, the CLI is deliberately positioned as the primary execution vector. Professional penetration testers and red teamers operate almost exclusively within terminal environments, frequently deploying tooling over restricted SSH sessions, pivot proxies, or reverse shells. A lightweight, text-based orchestrator integrates seamlessly into this operational reality, allowing for automated batch processing and rapid retargeting without the overhead of a graphical environment. Conversely, the Graphical User Interface (GUI) is provided as a comprehensive supplementary

option, designed primarily for post-engagement analysis, client reporting, and executive presentations where visualising the topological attack chain adds critical context.

To further bridge the gap between this automated pathfinding engine and established red-team workflows, a highly desirable future integration is the native generation of Neo4j Cypher queries. While project time constraints prevented its implementation in the current release, future iterations could automatically translate the neural network's output into executable Cypher syntax. Instead of merely logging a text-based path, the tool would generate a structured query instructing the Neo4j backend to explicitly match the exact sequence of SIDs and privilege edges identified by the beam search. This would empower operators to copy the generated string and instantly project the AI-discovered attack chain directly back into their native BloodHound interface for manual validation.

3.4.1 CLI

Commands and Interactions

The primary operational interface for the `GNN-AD-Navigator` is a comprehensive command-line orchestrator (`launch.sh`). Designed with efficiency and automation in mind, the CLI abstracts the underlying heterogeneous tensor transformations and Python micro-scripts, presenting the operator with a unified, parameterised execution environment.

Rather than forcing the user to manually execute individual scripts in sequence, the orchestrator dynamically routes execution based on the provided arguments. It structurally supports three distinct operational modes to accommodate various phases of a red-team engagement:

- **Data Preparation Only:** By supplying only the input and output directory paths, the CLI triggers the standalone ingestion pipeline. It compiles the raw, fragmented BloodHound and Certipy scans into the serialised `heterodata.pt` tensor and safely halts.
- **End-to-End Execution:** If an operator provides spatial coordinates alongside the directories (via the `-start` and `-target` flags), the orchestrator seamlessly transitions from data preparation directly into the neural inference phase, executing a full path discovery lifecycle in a single command.
- **Rapid Inference (Cache Bypass):** To optimise computational overhead during repetitive querying on the same Active Directory graph, operators can append the `-skip-prep` flag. This bypasses the heavy ingestion layer entirely, instantly executing the heuristic search against previously compiled tensors and returning results in a matter of seconds.

Beyond basic execution routing, the CLI exposes critical traversal hyperparameters directly to the operator. Through optional flags, users can dynamically dictate the neural architecture (e.g., `-model-type hgt`), supply custom pre-trained checkpoint weights (`-model`), and tune the determinism of the traversal algorithm by adjusting the beam search width (`-beam`) and the maximum exploration depth (`-depth`).

This parameterised design ensures the pipeline remains highly flexible. It allows security analysts to rapidly prototype, throttle, and evaluate different automated attack paths entirely from the terminal, without requiring any structural modifications to the underlying Python codebase.

3.4.2 GUI

Design and Theme

To ensure the tool remains accessible and operationally viable for enterprise environments, the GUI is implemented using the `Streamlit` framework. Deliberate user experience (UX) choices were made to distance the interface from the stereotypical, neon-heavy "hacker" aesthetic often found in offensive security tools. Instead, the application adopts a professional, office-app theme utilising a muted colour palette of navy blue (`#1f4e79`), slate, and maroon accents. This design rationale aligns with the tool's intended deployment context: a professional red-team or Security Operations Centre (SOC) environment where clarity, eye-strain reduction, and executive presentation are prioritised over flashy visuals.

Workflow Integration

The frontend is architected to serve as a seamless wrapper over the underlying command-line orchestrator. To facilitate user interaction, the interface incorporates a native `tkinter` directory picker with a text-input fallback, allowing operators to easily mount their BloodHound workspaces. When a pipeline execution is triggered, the GUI wraps the `launch.sh` orchestrator within a Python `subprocess`. Standard output is captured and streamed live directly to the interface, ensuring the operator retains full diagnostic visibility during data ingestion.

To optimise computational overhead, the application implements intelligent state management. It automatically detects the presence of pre-compiled artifacts (`forest_graph.json` and `heterodata.pt`) within the output directory, skipping the resource-intensive preparation phase if the data is already structured. Furthermore, the HGT model weights and flattened graph tensors are loaded into memory using Streamlit's `@st.cache_resource` decorators, reducing subsequent inference times from minutes to milliseconds.

Visualisation

Attack path outputs are rendered interactively using the `Pyvis` library. To prevent the chaotic, indistinguishable "hairball" structures common in large graph visualisations, the network is strictly constrained to a hierarchical, left-to-right layout with a fixed canvas height of 520 pixels. Nodes are colour-coded using the aforementioned muted palette to distinguish between Active Directory object types (e.g., users, groups, computers) at a glance.

Recognising that graphical representations can obscure technical nuances, the visualisation is paired with a deterministic, step-by-step table housed within a collapsible expander below the graph. This table explicitly maps the topological edges to their corresponding operational techniques (for example, mapping `AllowedToDelegate` to Constrained Delegation). Finally, to guarantee operational auditability, every execution is serialised and permanently persisted to disk as a timestamped inference log, ensuring that successful attack chains can be historically reviewed and integrated into formal red-team reporting.

Chapter 4

Experiments and Results

4.1 Experimental Setup

4.1.1 Test Environment: INLANEFREIGHT

To rigorously evaluate the operational efficacy of the `GNN-AD-Navigator`, inference testing was conducted against INLANEFREIGHT, an enterprise-scale Active Directory environment provided by Hack The Box (HTB) Academy. Designed to simulate a mature, complex corporate infrastructure, the environment spans multiple domains connected via cross-forest trust relationships. With a topology exceeding 4 000 nodes, it represents a dense, highly heterogeneous graph filled with realistic administrative noise, nested group structures, and complex GPO delegations, scaling significantly beyond the constrained boundaries of the training laboratory.

Crucially, the INLANEFREIGHT dataset was strictly isolated from the model during the training phase. Because the neural network weights were frozen prior to this evaluation, all pathfinding results generated within this forest represent genuine zero-shot generalisation. This environment serves as the definitive benchmark to prove that the HGT encoder successfully abstracted the underlying semantic logic of AD permissions—such as the mathematical relationship between a `GenericWrite` edge and a target `User` node—rather than merely memorising the static topological routes of the GOAD training corpus.

Data acquisition for both the training and testing environments was performed remotely using the `bloodhound-python` ingestor, with supplemental ADCS infrastructure scanned via `Certipy`. The pipeline dynamically ingested these disparate json outputs, stitched the certificate templates into the base graph, and resolved the global coordinate system dynamically, proving that the architecture can be deployed against novel environments without requiring structural modifications to the underlying PyTorch tensors.

However, the reliance on a Python-based collector introduces known topological limitations. Because `bloodhound-python` executes LDAP queries from a non-domain-joined Unix host, it inherently lacks the deep Remote Procedure Call (RPC) and Server Message Bloc (smb) enumeration capabilities natively enjoyed by the C#-based

SharpHound collector running on a compromised Windows endpoint. Consequently, the Python collector exhibits documented blind spots.

4.1.2 Diagnostic Environment: GOAD

While INLANEFREIGHT provides a realistic enterprise environment offers only for one targeted documented path to compromise, same goes for the child domain LOGISTICS.INLANEFREIGHT.LOCAL and the same forest trusted domain FREIGHTLOGISTICS.LOCAL each of them with a specific cross domain attack path. The domain works well for generalisation validation but it is insufficient for dissecting model behaviour in depth.

GOAD, despite being the training environment, serves a second purpose here as a diagnostic environment. Its multi-domain structure and richer set of documented attack paths make it better suited for probing the model's limitations, identifying where it fails and why. Queries run against GOAD in this context are not used to measure generalisation (the model has seen this environment) but rather to isolate specific architectural weaknesses under controlled conditions. Worth noting is that the path mentioned later are not provided in the training data.

4.1.3 Evaluation and Limitation Identification Logic

The evaluation methodology follows a continuous prompting approach: the model is repeatedly queried across different start and target node pairs, and its output is compared against BloodHound's built-in pathfinding on the same graph. The choices the model makes at each step, combined with knowledge of the pipeline's collected data and the `bloodhound-python` injector's known limitations, allow for precise isolation of each failure mode and a confident proposal of remediation actions for future work.

Queries are not filtered to show only successes. Failed queries, degenerate paths, and cases where the model's output diverges significantly from BloodHound's are reported alongside valid results. This approach avoids a cherry-picking reading of the results: the model is evaluated at its coverage boundary, and failures are treated as diagnostic data.

4.2 Results: wley → Domain Admin on INLANEFREIGHT

Query and Context

To establish a baseline for the model's navigational capabilities, the initial query targets a well-documented exploit chain within the `INLANEFREIGHT` enterprise environment. The objective is to navigate from the unprivileged user `wley` to achieve Domain Admin privileges. This scenario serves as a critical competency test: evaluating whether the

model can apply sequential privilege escalation mechanics learned in a small, 351-node laboratory (GOAD) to a structurally complex, unseen 4 000-node enterprise graph.

Recovered Path

Operating purely on its learned policy with an HGT confidence score of 0.5620, the tool correctly provides the documented path. Navigates from the unpriviledged user `wley` to reach the `domain admins` group. The tool given a new unseen Active Directory environment, navigates and recommendes the same documented path, traversing edges the same way it had learned during training on this new architecture.

1. **Password Reset:** `wley` executes a `ForceChangePassword` attack on the user `damundsen`.
2. **Targeted Write:** `damundsen` abuses `GenericWrite` privileges to compromise the `help desk level 1` group.
3. **Group Inheritance:** This access provides an implicit `MemberOf` transition into the `information technology` group.
4. **Control DCSync user:** The `information technology` group exercises `GenericAll` over the user `adunn`.
5. **Domain compromise:** The `user adunn` is connected via the ACL edge `GetChangesAll` to the domain controller, allowing it to dump the `NTDS.nit` database file.

Crucially, the pipeline's audit module halts the beam search precisely at `adunn`. It verifies that this user natively holds the required rights (`GetChanges` and `GetChangesAll`) to execute a DCSync attack on the target domain, terminating the search with confirmed operational success.

```

Path 1:
Score: 0.5620  Steps: 4
  [1] wley@inlanefreight.local
      -> damundsen@inlanefreight.local
      edge   : forcechangepassword
      attack : Password reset
      action : Force-change target's password
  [2] damundsen@inlanefreight.local
      -> help desk level 1@inlanefreight.local
      edge   : genericwrite
      attack : ACL abuse - targeted write
```

```
13          action : Modify SPN, msDS-
                AllowedToActOnBehalfOfOtherIdentity
14      [3] help desk level 1@inlanefreight.local
15          -> information technology@inlanefreight.local
16          edge   : memberof
17          attack : Group inheritance
18          action : Inherit rights via group membership
19      [4] information technology@inlanefreight.local
20          -> adunn@inlanefreight.local
21          edge   : genericall
22          attack : ACL abuse - full control
23          action : Reset password, add SPN, modify object
24
25    AUDIT:
26       Path terminates at a node with DCSync on the target domain
             'inlanefreight.local'. This is operational success.
```

Listing 4.1: model output for escalation path on INLANEFREIGHT

4.3 Result 2: Cross-Domain Trust Traversal

```
1  Start  : htb-student_adm@logistics.inlanefreight.local (users)
2  Target : domain admins@inlanefreight.local (groups)
3
4
5  Path 1:
6    Score: 0.2349  Steps: 6
7      [1] htb-student_adm@logistics.inlanefreight.local
8          -> domain admins@logistics.inlanefreight.local
9          edge   : memberof
10         attack : Group inheritance
11         action : Inherit rights via group membership
12     [2] domain admins@logistics.inlanefreight.local
13         -> logistics.inlanefreight.local
14         edge   : writedacl
15         attack : ACL abuse - DACL modification
16         action : Grant yourself GenericAll on target
17     [3] logistics.inlanefreight.local
18         -> inlanefreight.local
19         edge   : sameforesttrust
20         attack : Same-forest trust traversal
```

```
        action : Cross domain via SID History
    [4] inlanefreight.local
        -> freightlogistics.local
        edge   : crossforesttrust
        attack : Cross-forest trust traversal
        action : Inter-forest pivot
    [5] freightlogistics.local
        -> domain controllers@freightlogistics.local
        edge   : contains
        attack : Container hierarchy
        action : Object lives inside OU/container
    [6] domain controllers@freightlogistics.local
        -> academy-ea-dc03.freightlogistics.local
        edge   : contains
        attack : Container hierarchy
        action : Object lives inside OU/container

    AUDIT:
    X Path terminates at 'academy-ea-dc03.freightlogistics.
       local' which has no DCSync rights
    and is not the target. The model could not find a working
       attack chain.
    ! Final node is in domain 'academy-ea-dc03.freightlogistics
       .local' but target is
    in 'inlanefreight.local'. Cross-domain attack may need
       additional steps.
```

Listing 4.2: model output for cross-domain trust traversal on INLANEFREIGHT

The model navigated the cross domain trust edge, recommending the correct documented path. The processed graph portrays correctly the trust between domains. The model identified the edge type seen during training and traverses it. The rest of the path is noise a direct result of an architectural limitation. The specified target node `domain admins@inlanefreight.local` does not have a direct incoming path. When all the nodes are outgoing the model panics and produces noise. The audit script catches the errors and alerts the user.

4.3.1 Cross-Forest Foreign MemberOf

```

Start  : administrator@inlanefreight.local (users)
Target : administrators@freightlogistics.local (groups)
```

```
4
5  Path 1:
6    Score: 0.0918  Steps: 1
7      [1] administrator@inlanefreight.local
8          -> administrators@freightlogistics.local
9          edge   : memberof
10         attack : Group inheritance
11         action : Inherit rights via group membership
12
13   AUDIT:
14      Path reaches the requested target directly.
```

Listing 4.3: Model output on unseen node

One common misconfiguration in Active Directory forests, when the team with admin rights responsible for Active directory and network configurations, gives the admins of two domains membership of the same group. The admin account of one domain now holds admin rights over the other domain, allowing for easier interventions for the admin to adjust any configuration within the other domain in the same forest. The pipeline manages to extract the `cross-forest Foreign MemberOf` edge presenting it to the model. Once seen during training, the tool outputs the correct ACL abuse path to the user.

4.3.2 The "Ghost Edge" Anomaly: Raw Tensors against GUI Heuristics

As established in the experimental setup, the GOAD laboratory is utilised not to measure generalisation, but to conduct deep diagnostic evaluations of the model's architectural behaviour using attack paths deliberately excluded from the training corpus. During one such diagnostic query, the heuristic beam search repeatedly identified a high-confidence attack path relying on a `MemberOf` transition from a compromised computer object (`winterfell.north.sevenkingdoms.local`) to a highly privileged, group (`enterprise domain controllers@sevenkingdoms.local`).

However, when this exact operational route was manually verified within the official BloodHound graphical user interface, the final `MemberOf` edge was completely absent. This created an apparent contradiction between the neural network's successful traversal and the industry-standard visualisation tool's failure to render the path.

```
1  Path 1:
2    Score: 0.9745  Steps: 2
3      [1] castelblack.north.sevenkingdoms.local
4          -> winterfell.north.sevenkingdoms.local
```

```
        edge    : allowedtodelegate
        attack : Constrained Delegation (S4U2Self + S4U2Proxy
           impersonation)
    [2] winterfell.north.sevenkingdoms.local
        -> enterprise domain controllers@sevenkingdoms.local
        edge    : memberof
        attack : Group inheritance (Inherit rights via group
           membership)

AUDIT:
   Path terminates at node with DCSync on target domain '
      kingslanding.sevenkingdoms.local'. Operational success.
   Path ends structurally with 'memberof' edge. Likely noise
      from beam search.
```

Listing 4.4: Model output on unseen node

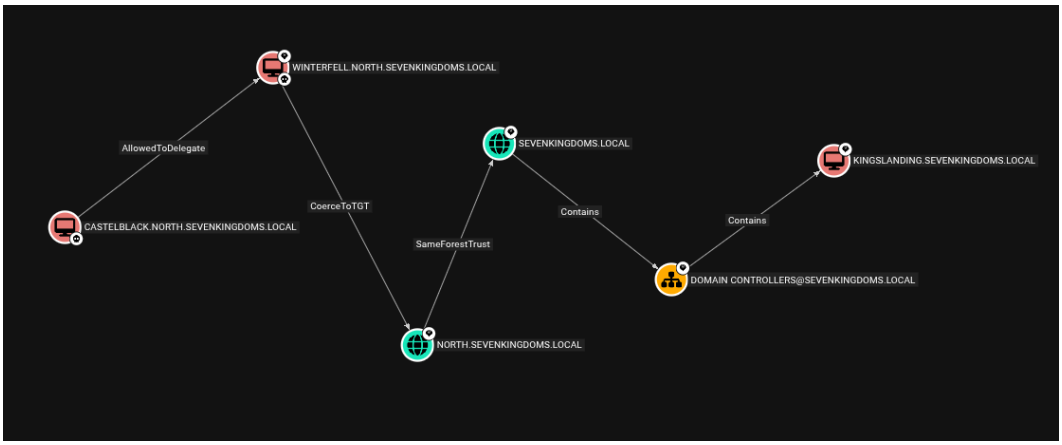

Figure 4.1: BloodHound missing the memberOf edge and not finding the actual shortest path.

To deterministically resolve this discrepancy, a manual audit of the pipeline's underlying data structures was conducted. By mapping the objects to their (SIDs)—specifically, the source computer (SID: `S-1-5-21-2466621736-3584596050-225777411-1001`) and the target group (SID: `SEVENKINGDOMS.LOCAL-S-1-5-9`)—a programmatic search of the compiled (PyG) `HeteroData` tensor was executed. The audit definitively returned a positive match within the `('computers', 'memberof', 'groups')` edge index, proving the transition mathematically exists within the collected ground truth.

The absence of this edge within the BloodHound GUI highlights a critical vulnerability in relying exclusively on presentation-layer tools. While the backend Neo4j database likely ingested the relationship from the raw `bloodhound-python` scans, the BloodHound frontend employs restrictive Cypher query heuristics and visual filtering.

To prevent browser memory exhaustion and UI clutter (the "hairball" effect), the application frequently truncates cross-domain nested group memberships or excludes universal well-known SIDs from its default shortest-path calculations.

This anomaly demonstrates a significant operational advantage of deploying deep learning directly against the raw tensor structure. The `GNN-AD-Navigator` operates exclusively on the unrolled mathematical matrices, it is entirely immune to the visual filtering constraints and hardcoded query limitations of frontend applications. It successfully surfaces stealthy, structurally valid "ghost edges" that a human operator relying solely on standard visualisation interfaces would completely overlook.

4.4 Result 3: Constrained Delegation

```
Start  : castelblack.north.sevenkingdoms.local (computers)
Target : kingslanding.sevenkingdoms.local (computers)

Path 1:
  Score: 1.0000  Steps: 2
    [1] castelblack.north.sevenkingdoms.local
        -> winterfell.north.sevenkingdoms.local
        edge   : allowedtodelegate
        attack : Constrained Delegation
        action : S4U2Self + S4U2Proxy impersonation
    [2] winterfell.north.sevenkingdoms.local
        -> enterprise domain controllers@sevenkingdoms.local
        edge   : memberof
        attack : Group inheritance
        action : Inherit rights via group membership

  AUDIT:
     Path terminates at a node with DCSync on the target domain
         'kingslanding.sevenkingdoms.local'. This is operational
         success.
```

Listing 4.5: Model output on unseen node

A key insight to foreground is that the `AllowedToDelegate` edge was missed during the graph construction during training. The `AllowedToDelegate` is an edge type not directly present in the raw BloodHound scan; it needed further processing, deducing the relationship between the two nodes before establishing its existence. Hence, during training, the model did not see this edge type. When introduced for this query as the only navigable node, the model successfully suggested traversing it and gave a score of

1.0 (the only probable path).

Notably, this phenomenon was not isolated to constrained delegation; the exact same behaviour was observed with the `CoerceToTGT` edge, which was also successfully traversed despite being invisible during the training phase. Furthermore, this dynamic highlights a structural advantage of the pipeline: by manually modifying the heuristic weights of these once-invisible edges during inference, an operator can directly affect the model's navigation. Tuning these weights forces the beam search to either prioritise or penalise these specific, newly-discovered attack techniques based on the red team's operational context.

4.5 Result 4: Observed Experimental Constraints

While the zero-shot evaluation on INLANEFREIGHT demonstrated significant operational capability, the diagnostic queries also exposed structural limitations within the pipeline. These limitations stem primarily from the nature of heuristic beam search algorithms and the distribution of the supervised training data.

4.5.1 Heuristic Divergence and Terminal State Blindness

A query from `castelblack.north.sevenkingdoms.local` to `braavos.essos.local` produced an interesting topological divergence between the model's output and BloodHound's native pathfinding.

The `GNN-AD-Navigator` navigated the following route: `castelblack` → `winterfell` (`AllowedToDelegate`) → `enterprise domain controllers@sevenkingdoms` (`MemberOf`) → `sevenkingdoms.local` (`GetChanges`, partial DCSync) → `essos.local` (`CrossForestTrust`) → `laps@essos` (`Contains`) → `braavos.essos.local` (`Contains`).

Conversely, BloodHound identified a structurally different route passing through `CoerceToTGT` on `north.sevenkingdoms.local`, followed by a `SameForestTrust` pivot and an ACL chain through `tyron.lannister` (`GenericWrite`) and `ReadLAPSPassword` toward the target.

This divergence is explained by two factors. First, while the neural network demonstrated the ability to traverse `CoerceToTGT` edges in isolation (as seen in Section 4.4), its policy head naturally biased toward the mathematically dominant `CrossForestTrust` vector because it strongly resembles the primary domain-hopping techniques seen during training. Second, the two `Contains` hops at the end of the model's path represent structural noise. The model successfully reached `essos.local` and acquired DCSync rights, but it continued walking down the container hierarchy rather than halting and recommending impersonation. This behaviour explicitly reflects the "terminal state blindness" discussed earlier, where the model struggles to recognise when sufficient privileges have been acquired to conclude the attack.

Neither path is strictly superior. The model relies on topological trust traversal, while BloodHound relies on protocol-level coercion.

4.5.2 Depth Constraints and Sub-Querying

A fundamental limitation of deploying a beam search algorithm on a graph exceeding 4 000 nodes is the constraint of maximum exploration depth. During testing, a highly complex, stealthy attack route requiring over 10 distinct transitions resulted in a pipeline failure; the model returned no valid paths. This occurs because the beam search aggressively prunes low-probability branches early in the execution to save memory. If the initial steps of a long attack chain appear mathematically disadvantageous, the model discards the route before reaching the objective.

To prove that the neural architecture possessed the requisite topological knowledge, the objective was manually chunked into three sequential sub-queries, effectively providing the model with operational waypoints. As demonstrated in Figure 4.2, the model successfully routed each of the three smaller segments with maximum confidence. This confirms that while the deep mathematical relationships are successfully embedded within the HGT tensors, single-shot execution over exceptionally long horizons remains a limitation of the current beam configuration.

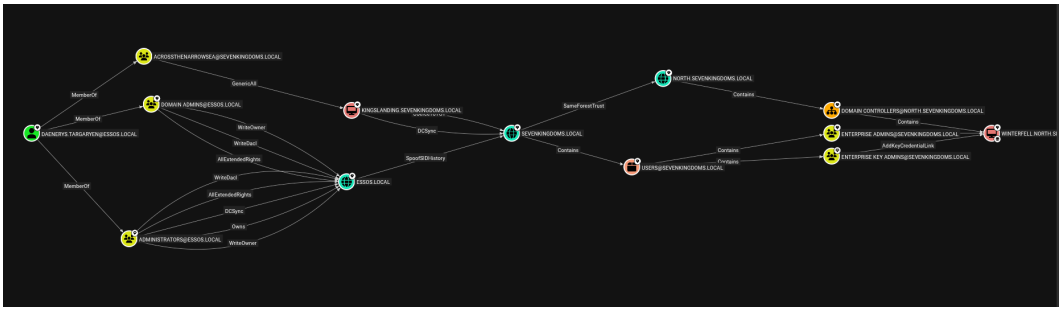

Figure 4.2: The shortest path between the two nodes is 6 hopes

```
Start  : daenerys.targaryen@essos.local (users)
Target : winterfell.north.sevenkingdoms.local (computers)

============================================================
RESULTS — top 3 path(s)
============================================================

Path 1:
  Score: 0.0478  Steps: 6
    [1] daenerys.targaryen@essos.local
        → domain admins@essos.local
        edge    : memberof
```

```
13          attack : Group inheritance (Inherit rights via group
                membership)
14      [2] domain admins@essos.local
15          → default domain controllers policy@essos.local
16          edge   : owns
17          attack : Object ownership (Modify DACL of owned object)
18      [3] default domain controllers policy@essos.local
19          → domain controllers@essos.local
20          edge   : gplink
21          attack : Unknown edge (Manual review)
22      [4] domain controllers@essos.local
23          → meereen.essos.local
24          edge   : contains
25          attack : Container hierarchy (Object lives inside OU/
                container)
26      [5] meereen.essos.local
27          → enterprise domain controllers@essos.local
28          edge   : memberof
29          attack : Group inheritance (Inherit rights via group
                membership)
30      [6] enterprise domain controllers@essos.local
31          → windows authorization access group@essos.local
32          edge   : memberof
33          attack : Group inheritance (Inherit rights via group
                membership)
34
35    AUDIT:
36      × Path terminates at 'windows authorization access
          group@essos.local' lacking DCSync and is not the target.
```

Listing 4.6: Pipeline execution failure due to structural noise and terminal state blindness

As discussed, attempting to navigate the complete topological distance from the initial foothold to the final objective in a single inference execution frequently triggers heuristic pruning failures. Listing 4.8 demonstrates this behaviour. The beam search algorithm discards the optimal, stealthy route early in the execution, instead favouring highly-rewarded intra-domain edges that ultimately trap the model within structural container noise and result in an operational audit failure.

```
1 Start  : daenerys.targaryen@essos.local (users)
2 Target : north.sevenkingdoms.local (domains)
3
4 ============================================================
```

```
RESULTS — top 1 path(s)
============================================================

Path 1:
  Score: 0.0014  Steps: 5
    [1] daenerys.targaryen@essos.local
        → domain admins@essos.local
        edge   : memberof
        attack : Group inheritance (Inherit rights via group
           membership)
    [2] domain admins@essos.local
        → default domain policy@essos.local
        edge   : owns
        attack : Object ownership (Modify DACL of owned object)
    [3] default domain policy@essos.local
        → essos.local
        edge   : gplink
        attack : Unknown edge (Manual review)
    [4] essos.local
        → sevenkingdoms.local
        edge   : crossforesttrust
        attack : Cross-forest trust traversal (Inter-forest
           pivot)
    [5] sevenkingdoms.local
        → north.sevenkingdoms.local
        edge   : sameforesttrust
        attack : Same-forest trust traversal (Cross domain via
           SID History)

  AUDIT:
    ✓ Path reaches the requested target directly.
```

Listing 4.7: The tool provides the first half of the path correctly

To definitively prove that the underlying HGT embeddings possess the requisite topological knowledge to solve this route, the objective was manually segmented to bypass the beam depth constraints. Listing **??** illustrates the first of these segmented waypoints. By breaking the path, the model successfully navigates the complex multi-forest trust boundary, bridging the Essos domain to the North domain over a precise five-step horizon.

```
Start  : north.sevenkingdoms.local (domains)
Target : winterfell.north.sevenkingdoms.local (computers)
```

```
3
4  ============================================================
5  RESULTS — top 1 path(s)
6  ============================================================
7
8  Path 1:
9    Score: 0.1430  Steps: 2
10     [1] north.sevenkingdoms.local
11         → domain controllers@north.sevenkingdoms.local
12         edge   : contains
13         attack : Container hierarchy (Object lives inside OU/
               container)
14     [2] domain controllers@north.sevenkingdoms.local
15         → winterfell.north.sevenkingdoms.local
16         edge   : contains
17         attack : Container hierarchy (Object lives inside OU/
               container)
18
19   AUDIT:
20     ✓ Path reaches the requested target directly.
```

Listing 4.8: The model correctly complets the path

Having successfully resolved the macro-architecture of the cross-forest trust, a final, highly-constrained sub-query was issued to close the remaining topological distance to the objective. As demonstrated in Listing **??**, providing the model with this localized waypoint allows it to effortlessly traverse the final container hierarchy, landing directly on the target computer object with maximum precision.

4.5.3 ADCS Feature Scarcity

Finally, while the data pipeline successfully stitches ADCS attributes (via Certipy) into the PyG tensors, the model struggles to prioritise stealthy, deep certificate escalation chains (such as ESC1 or ESC8). This is a direct consequence of the supervised training data distribution.

Within the GOAD training corpus, traditional ACL abuses (`GenericAll`, `WriteDacl`) and group inheritances are vastly overrepresented compared to certificate template exploitation. Because neural networks inherently optimise for the most frequently rewarded patterns, the policy head is mathematically biased against ADCS transitions. Unless the certificate edge is the absolute only viable path forward, the beam search will almost always favour a traditional RPC or Lightweight Directory Access Protocol (LDAP) privilege escalation vector. Addressing this imbalance would require generating

a highly specialised, ADCS-heavy training dataset in future iterations.

4.5.4 Heuristic Pruning of Cross-Boundary Edges

While sub-querying proved effective for bypassing depth constraints, diagnostic testing revealed a secondary vulnerability within the inference engine: the aggressive heuristic pruning of Out-of-Distribution (OOD) topological features. This was explicitly observed during a cross-forest evaluation originating from `daenerys.targaryen@essos.local` and targeting `kingslanding.sevenkingdoms.local`.

Topologically, a highly efficient, two-step attack path exists between these nodes. The user possesses a cross-forest foreign group membership to `acrossthenarrowsea@sevenkingdoms.local`, which in turn holds `GenericAll` privileges over the target computer. However, as demonstrated in Listing 4.9, when executed as a single end-to-end query, the model completely bypassed this optimal route, instead favouring a convoluted six-step path that ultimately failed due to terminal state blindness.

```
Start  : daenerys.targaryen@essos.local (users)
Target : kingslanding.sevenkingdoms.local (computers)

Path 1:
  Score: 0.0478  Steps: 6
    [1] daenerys.targaryen@essos.local
        → domain admins@essos.local
        edge   : memberof
    [2] domain admins@essos.local
        → default domain controllers policy@essos.local
        edge   : owns
    [3] default domain controllers policy@essos.local
        → domain controllers@essos.local
        edge   : gplink
    [4] domain controllers@essos.local
        → meereen.essos.local
        edge   : contains
    [5] meereen.essos.local
        → enterprise domain controllers@essos.local
        edge   : memberof
    [6] enterprise domain controllers@essos.local
        → windows authorization access group@essos.local
        edge   : memberof

  AUDIT:
```

```
26    × Path terminates at 'windows authorization access
        group@essos.local' lacking DCSync and is not the target.
```

Listing 4.9: The model bypassing the optimal two-step route due to early beam pruning.

To diagnose this failure, the route was manually broken down into two isolated sub-queries. The output revealed that the neural policy head assigned a mathematical confidence score of exactly `0.0000` to the initial cross-forest `MemberOf` transition. Because foreign group memberships were statistically rare within the GOAD supervised training dataset, the network heavily penalised the structural anomaly of a user in Domain A linking directly to a group in Domain B.

This exposes a fundamental limitation of the Beam Search algorithm in offensive security applications. Because the search is inherently greedy, it evaluates cumulative probabilities at each depth step. The `0.0000` confidence score on the very first transition caused the algorithm to instantly prune the branch in favour of standard, intra-domain edges (such as the jump to `domain admins@essos.local`), permanently blinding the model to the guaranteed compromise (`Score: 1.0000`) waiting just one hop further down the cross-forest chain.

The mathematical origin of this `0.0000` confidence score lies in the interaction between the supervised training distribution and the HGT embedding space. The HGT encoder constructs node embeddings by aggregating local neighbourhood features. Consequently, nodes residing in disparate domains are projected into distinct, distant clusters within the high-dimensional latent space.

To formalise this, let h_u represent the latent embedding of the source node (e.g., the Essos-based user) and h_v represent the embedding of a candidate next-hop node. As defined in the architecture, the policy head calculates a raw transition logit z_v by passing the element-wise (Hadamard) product of these embeddings through a Multi-Layer Perceptron (MLP):

$$z_v = \text{MLP}(h_u \odot h_v) \tag{4.1}$$

Because the curated training corpus heavily emphasised intra-domain group inheritance, the network never received positive gradient reinforcement for cross-forest relationships. Thus, the interaction between the distant embeddings of the Essos user and the SevenKingdoms group yields a comparatively low raw logit (z_{cross}).

During the final traversal evaluation, the raw logits of all valid neighbour transitions $\mathcal{N}(u)$ are passed through a Softmax activation function to generate a normalised traversal probability distribution $P(v|u)$:

$$P(v|u) = \frac{\exp(z_v)}{\sum_{k \in \mathcal{N}(u)} \exp(z_k)} \tag{4.2}$$

This equation acts as a rigorous competitive filter. When the model evaluates the highly familiar, explicitly rewarded intra-domain edge (the jump to `domain admins@essos.local`), it produces a significantly larger logit (z_{intra}). Because the Softmax function exponentiates these logits, the large value of $\exp(z_{intra})$ completely dominates the denominator. This mathematical mechanism exponentially suppresses the lower cross-forest logit, effectively squashing its traversal probability to exactly `0.0000` and causing the Beam Search algorithm to instantly prune the optimal route.

4.6 Discussion

4.6.1 Stateful Navigation and the Coverage Ceiling

The curation of the GOAD training dataset established a strict operational boundary for the model. From the raw execution logs, approximately 49 distinct transitions were documented, which were subsequently filtered down to 33 purely topological, learnable attack vectors. This finite vocabulary directly influenced the fundamental architecture of the system.

Initial iterations of this research explored a "stateful" or context-aware policy head. This approach would have required the model to maintain an active inventory of compromised credentials and privileges dynamically during the beam search, allowing its navigation to be conditioned on acquired assets rather than purely spatial topology. However, implementing a recurrent or reinforcement-based state tracker required a massively expanded, dynamic training dataset that far exceeded the 33 static traces available.

Consequently, the architecture was constrained to stateless topological inference. The model successfully learns the structural semantics of techniques like ACL abuse and DCSync, but it cannot logically chain attacks that require holding a specific credential from Step A to execute Step D. The development of dynamic attack inventory generation is a natural evolution of this architecture but requires a distinct reinforcement learning objective and is explicitly designated as future work.

4.6.2 Pipeline Dominance: Decoupling Data from Weights

Perhaps the most significant architectural observation from the inference testing is the model's handling of completely unseen edge types. During training, the neural network never encountered `AllowedToDelegate` or `CoerceToTGT` primitives. Yet, once the Python data pipeline (`stitcher.py` and `build_dataset.py`) was updated to extract and synthetically generate these relationships, the frozen model successfully traversed them during zero-shot inference without requiring any mathematical retraining.

This behaviour proves that the model's ability to discover novel attack paths is

heavily dictated by the completeness of the data ingestion pipeline. Because the neural network relies on the graph's physical topology to evaluate contiguous sub-graphs, introducing a newly discovered AD vulnerability simply translates to injecting a new structural pathway into the PyG tensor. Even without learned attention weights explicitly tuned for `CoerceToTGT`, the policy head opted to traverse it because it represented a mathematically viable bridge toward the target embedding.

This decoupling of graph coverage from neural capability is a massive operational advantage. It implies that security teams do not need to recompile or retrain the deep learning architecture from scratch every time a novel AD misconfiguration is published to the red-team community. Instead, updating the upstream json parsers to map the new technique into the static graph is sufficient to immediately expand the AI's operational reach. For this reason, the deterministic data engineering pipeline developed in this research is considered a primary contribution, functioning as a critical enabler for the underlying neural network.

Chapter 5

Limitations and Future Work

While the empirical evaluation in Chapter 4 demonstrates that the Heterogeneous Graph Transformer HGT successfully learns and generalises offensive navigation policies, the current pipeline remains a proof of concept. Transitioning from a controlled laboratory environment to a production-ready red teaming tool requires an honest assessment of the system's boundaries. This chapter systematically details the fundamental mathematical, algorithmic, and operational limitations of the current approach, and proposes a concrete engineering roadmap for a second-generation architecture.

5.1 Mathematical and Algorithmic Limitations

The most significant constraints on the GNN-AD-Navigator do not stem from data ingestion or environmental scanning, but from the inherent mechanics of deep learning when applied to complex, asymmetrical graph structures. Understanding these mathematical flaws is critical for interpreting the model's behaviour during deep attack simulations.

5.1.1 Multiplicative Probability Decay and Search Bias

The inference phase relies on a heuristic beam search to navigate the Active Directory graph. At each step, the policy head outputs a Softmax probability distribution over all available candidate edges. The total confidence score for a given attack path is then calculated by multiplying the individual probabilities of each hop along that route.

Mathematically, multiplying fractions consistently yields a smaller product (e.g., $0.8 \times 0.8 \times 0.8 = 0.512$). In the context of Active Directory attack paths, this creates a phenomenon known as *multiplicative probability decay*. The search algorithm inherently penalises long attack chains simply because they require more mathematical transitions.

From an offensive security perspective, this mathematical decay directly contradicts the foundational objective of the navigation policy. The primary motivation for engineering transition-specific confidence scores—effectively acting as an OPSEC step cost—was to explicitly train the model to prioritise stealth over path efficiency. In professional red teaming engagements, operators heavily rely on LotL techniques, deliberately chaining

multiple low-privilege, native Active Directory misconfigurations to escalate privileges without triggering SIEM (Security Information and Event Management) alarms. The model was designed to recognise these stealthy, deep attack chains as operationally superior to a noisy, direct exploit. However, because the probability decay compounds with every additional transition, the mathematical penalty of path length eventually overwhelms the learned OPSEC rewards. As a result, the beam search prematurely prunes the desired stealthy chains, paradoxically forcing the model to revert to the exact behaviour it was engineered to avoid: demonstrating a strong preference for shorter, highly detectable attacks simply because they require fewer algorithmic multiplications.

5.1.2 Bounded Receptive Field and the Horizon Problem

The expressiveness of a Graph Neural Network is fundamentally tied to its receptive field, which is dictated by the number of hidden layers (K). During the message-passing phase, a node aggregates feature data from K hops away. Therefore, to intrinsically understand the structural relationship between a compromised starting user and a target Domain Controller seven steps away, the model would theoretically require seven layers of message passing.

However, Active Directory environments are highly dense, "small world" networks, characterised by heavily nested group memberships and extensive permissions. In such dense topologies, expanding the depth of the network triggers a well-documented graph learning limitation known as *over-smoothing*. If the Heterogeneous Graph Transformer HGT is configured with too many layers, the repeated aggregation of neighbourhood features causes the embeddings of structurally distinct nodes—such as a standard workstation and an Enterprise Admin group—to mathematically converge toward indistinguishable mean vectors.

To preserve the distinct privilege hierarchies required for accurate routing, the GNN-AD-Navigator's architecture is strictly bounded to a shallow depth (specifically $K = 2$ layers, as established in the methodology). While this successfully prevents over-smoothing and maintains structural fidelity, it directly introduces the *horizon problem*. When executing a deep attack simulation (e.g., a 7-hop path), the starting node's embedding contains zero latent information about a target that lies beyond its two-hop receptive field. The policy head is effectively blind to the distant destination, forcing the algorithm to navigate using only local privilege gradients rather than true, end-to-end topological awareness.

5.1.3 The "Blind Compass" Effect in Lateral Movement

As detailed in Chapter 2, a critical engineering adaptation within the pipeline was the design of the policy head. Rather than computing a target-aware concatenation, the

model utilises the Hadamard product (element-wise multiplication) between the current node's embedding and the candidate neighbour's embedding. This architectural choice proved highly effective for the primary goal of Active Directory exploitation: privilege escalation. By forcing the model to evaluate the absolute semantic value of a transition, it successfully learned a generalised privilege hierarchy. It acts as a "universal climber" that inherently gravitates toward high-value nodes like Domain Admins, regardless of the specific topology.

However, this adaptation introduces a distinct operational trade-off. Because the target node's embedding is mathematically excluded from the transition scoring, the policy head possesses no "target gravity." The model implicitly knows how to escalate privileges, but it does not inherently know its final destination.

This limitation becomes acutely apparent during horizontal lateral movement. If an attack simulation requires navigating through a flat network segment— such as pivoting between multiple standard workstations within an isolated Organizational Unit (OU)— the model struggles to find an optimal route. When presented with candidate neighbours that share identical, low-privilege weights, the policy head lacks the directional context required to efficiently break the mathematical tie. Without the target vector actively pulling the algorithm toward a specific coordinate, the search can oscillate or wander inefficiently. The model effectively acts as a "blind compass"—perfectly calibrated to point "up" toward domain dominance, but unable to navigate laterally toward a specific, non-administrative target.

5.2 Operational and Data Boundaries

A fundamental axiom of machine learning is that a model's operational capacity is strictly bounded by the fidelity of its underlying data. While the algorithmic limitations define how the model navigates the graph, the operational boundaries dictate whether the necessary topological pathways exist in the underlying dataset to begin with.

5.2.1 Edge Directionality and the Computer Node Sinkhole

During the empirical evaluation, the model occasionally exhibited suboptimal routing by bypassing topologically logical hops through specific Computer nodes. This behaviour is not an algorithmic failure, but rather a direct consequence of how `PyTorch Geometric` enforces strict edge directionality, combined with the inherent nature of Active Directory Access Control Lists (ACLs).

In a standard BloodHound-compatible graph, privileges are mapped as directed edges from a principal (the controller) to a target (the controlled). Consequently, Computer nodes predominantly accumulate incoming edges (e.g., Users or Groups holding `GenericAll` or `AdminTo` rights over the machine) but possess virtually zero

outgoing rights over other objects in the domain. Without the ability to dynamically resolve and traverse active user sessions (e.g., using a `HasSession` edge to pivot from the compromised machine to a highly privileged Domain Admin logged into it), the Computer node becomes a topological sinkhole.

The inference engine mathematically learns to avoid these nodes because traversing into them almost always results in a dead end. While a human operator would compromise the computer, dump memory credentials (e.g., via Mimikatz), and pivot, the current graph schema does not explicitly represent that post-exploitation capability as a traversable outgoing edge.

5.2.2 Collector Incompleteness and Schema Evolution

The current data pipeline relies heavily on the Linux-compatible Python ingestor (`bloodhound-ce-python`). While this ensures the toolset remains accessible from standard UNIX-based attack hosts, it introduces severe structural blind spots. Most notably, the Python collector exhibits incompleteness regarding the enumeration of explicit Trust accounts. Furthermore, it lacks native support for Active Directory Certificate Services (ADCS), necessitating reliance on legacy versions of `Certipy` and complex logic to stitch the graph together.

This fragility was further exacerbated by environmental volatility during graph enumeration phases. For example, hypervisor kernel incompatibilities and virtual network bridging failures during the provisioning of the `ESSOS.local` test domain demonstrated that relying on fragmented, multi-tool data collection is difficult to scale consistently.

Ultimately, this is an ingestion constraint, not a model limitation. For a production-grade deployment, future iterations must deprecate the Python ingestor in favour of the official C# SharpHound collector. Executing SharpHound natively within the target environment would inherently surface coercion-derived edges, complete trust mappings, and richer ACL enumeration, directly expanding the model's discoverable attack surface without requiring any architectural modifications to the neural network.

5.2.3 ADCS Edge Sparsity and Static Stitching

The current dataset exhibits severe class imbalance regarding ADCS attack primitives. Within the training corpus, ADCS template transitions—such as `Enroll` or `WriteDacl` over a `CertTemplate` node—are extremely scarce, typically limited to only one or two traversable instances per template.

From a deep learning perspective, this edge sparsity presents a significant challenge for the HGT. Neural networks require sufficient statistical frequency to accurately learn the latent semantic weight of a transition. Because the model observes thousands of

`MemberOf` or `GenericAll` relationships but only a handful of ADCS escalations, the policy head inherently struggles to assign an operationally confident probability score to certificate-based attacks during inference, occasionally causing the beam search to undervalue them.

Furthermore, this sparsity is compounded by the pipeline's ingestion mechanics. Because the Python collector natively lack comprehensive ADCS enumeration, these specific edges are currently extracted via fragmented `Certipy` scans and statically stitched into the underlying graph using custom Python logic. While this static injection successfully bridges the topological gap and physically allows the model to traverse the ADCS infrastructure, the hardcoded nature of the stitching process remains brittle. Future iterations must address this data scarcity—either through the injection of synthetic ADCS traces to artificially amplify the training signal, or by adopting a unified, native collection tool that reliably surfaces certificate misconfigurations at scale.

5.3 Future Work: Algorithmic Upgrades

The mathematical constraints identified in Section 5.1 cannot be resolved simply by increasing the volume of training data or arbitrarily tuning hyperparameters. Addressing the horizon problem and the adverse effects of multiplicative probability decay requires fundamental architectural enhancements to the inference engine. The following algorithmic upgrades outline a clear engineering roadmap for a second-generation navigation tool, focusing on hybridizing deep learning embeddings with deterministic routing logic.

5.3.1 Waypoint Routing and Path Segmentation

To overcome the bounded receptive field of the Heterogeneous Graph Transformer, future iterations of the pipeline should implement a "divide-and-conquer" routing wrapper. As previously established, the model struggles with deep attack chains because targets beyond a two-hop radius fall completely outside the starting node's latent awareness, creating a blind horizon.

A waypoint routing architecture would solve this by decoupling the global pathfinding strategy from the local transition predictions. Much like modern GPS navigation systems, a static pre-processor could analyse the global graph topology to identify major structural hubs or "choke points"— such as highly connected OU, Tier-0 infrastructure, or centralized identity providers.

Instead of querying the neural network to calculate a continuous, 10-hop path from a low-privileged user directly to a Domain Controller, the wrapper would segment the objective. The model would first be tasked with navigating to the nearest waypoint; upon success, the search state would reset and generate a new trajectory to the subsequent waypoint.

This segmentation directly neutralises both the horizon problem and the multiplicative probability decay. By ensuring that each individual path segment remains short, the intermediate targets stay within the model's receptive field. Furthermore, resetting the probability calculation at each waypoint prevents the accumulated mathematical penalties from overwhelming the learned OPSEC step costs, allowing the model to freely select stealthy, non-linear routes.

5.3.2 Guided Search Heuristics (A^* Algorithm)

While waypoint routing mitigates the horizon problem, the fundamental inference mechanism must also evolve to protect OPSEC-driven routing. The current implementation relies on a heuristic beam search, which, as established, falls victim to multiplicative probability decay. To preserve long, stealthy attack chains, future architectures should replace the pure beam search with an additive, cost-based heuristic algorithm such as A^* (A-Star).

Unlike beam search, which evaluates routes by multiplying fractional Softmax probabilities, A^* evaluates candidate paths using an additive cost function: $f(n) = g(n)+h(n)$. In this paradigm, $g(n)$ represents the exact, accumulated OPSEC cost of the path taken so far, while $h(n)$ serves as the heuristic estimating the remaining distance or effort to the target. By summing step costs rather than multiplying probabilities, the mathematical penalty for path length grows linearly rather than exponentially.

This algorithmic shift would elegantly repurpose the existing deep learning architecture. Instead of acting as a standalone, greedy pathfinder, the HGT policy head would be deployed exclusively as the heuristic function, $h(n)$. The GNN would evaluate candidate neighbours and assign dynamic, context-aware proximity scores based on learned privilege hierarchies, while the deterministic A^* framework strictly tracks the accumulated stealth metrics ($g(n)$).

This hybrid approach ensures that operationally superior, low-privilege LotL chains are evaluated on their true tactical merit. It mathematically protects longer paths from being prematurely pruned, completely eradicating the current system's bias toward short, highly detectable exploit paths.

5.3.3 Terminal State Learning and Global Reachability

The current inference engine relies on a hardcoded symbolic predicate to identify terminal states: if the search algorithm lands on a node possessing DCSync rights (e.g., `GetChanges` and `GetChangesAll` on the target domain), the search halts, assuming total domain compromise. While theoretically rudimentary, the empirical results from Chapter 4 demonstrate that this static cutoff condition is operationally sufficient and highly effective for current red teaming engagements.

A theoretically purer approach would involve replacing this hardcoded heuristic with Offline Reinforcement Learning (RL). By training a dedicated value head over the existing traces using algorithms like Conservative Q-Learning (CQL), the model could intrinsically learn to assign massive reward values to DCSync-capable nodes. However, deploying an Offline RL architecture is exceptionally complex, computationally expensive, and prone to severe training instability. Given the high performance of the current static condition, introducing a brittle RL pipeline represents significant engineering overhead for diminishing returns.

A more pragmatic, data-driven alternative exists to teach the policy head the concept of global reachability without altering the underlying architecture. The training corpus could be augmented with synthetic traces where reaching a DCSync-capable node is immediately followed by a direct transition to the final target, regardless of whether a literal topological edge exists. By exposing the model to these synthetic "teleportation" transitions, the HGT would mathematically learn that owning a DCSync node equates to owning the entire graph. This allows the existing architecture to organically learn the semantic weight of domain dominance entirely through data representation, bypassing the need for a resource-heavy Reinforcement Learning overhaul.

5.3.4 LLM Integration for Out-Of-Vocabulary (OOV) Edges

As offensive enumeration tools like SharpHound evolve, their output schemas and edge vocabularies frequently change (e.g., introducing new attack primitives or altering nomenclature such as `WriteDACL` to `WriteDacl`). The current pipeline relies on a static alias mapping dictionary to normalise these relationships before tensor vectorisation. From an engineering perspective, this static approach is highly efficient, computationally lightweight, and operationally sufficient for the current Active Directory landscape.

However, maintaining a hardcoded vocabulary introduces long-term technical debt and fragility when encountering undocumented, OOV edges during a live engagement. A promising future direction is the integration of a localized, lightweight Large Language Model (LLM) to act as a semantic parser at the ingestion layer. Instead of dropping or ignoring unknown relationships, the LLM could semantically evaluate the new edge type and automatically map it to the closest known privilege vector in the model's latent space. While static mappings are entirely sufficient for today's red teaming operations, LLM-assisted parsing would provide a highly resilient, future-proof ingestion pipeline capable of organically adapting to evolving Active Directory schemas.

5.4 Conclusion

The limitations and proposed upgrades outlined in this chapter do not diminish the empirical success of the current `GNN-AD-Navigator`; rather, they define the precise

engineering trajectory required to transition from a constrained proof-of-concept to an enterprise-grade autonomous agent.

An application that successfully surpasses these boundaries—coupling the deep semantic embeddings of a Heterogeneous Graph Transformer with the deterministic routing of an A^* heuristic and localised waypoint segmentation—would represent a paradigm shift in automated offensive security. Such a system would prove that AI-driven pathfinding can scale to massive, asymmetrical Active Directory environments without sacrificing the OPSEC requirements crucial for advanced LotL attack chains. It would effectively transition automated red teaming from rigid, shortest-path vulnerability scanners into dynamic, stealth-conditioned navigators.

More broadly, this research roadmap redefines the role of Graph Neural Networks in the cybersecurity domain. Historically relegated to defensive classification tasks, such as anomaly detection or alert clustering, this work establishes GNNs as foundational reasoning engines for offensive operations. By proving that neural networks can mathematically abstract and internalise the semantic hierarchy of privilege escalation, it lays the theoretical groundwork for a new generation of agentic AI—tools capable of navigating complex digital topologies with the intuitive, contextual awareness of a human Red Teamer.

General Conclusion

This thesis presented the design, implementation, and empirical evaluation of the `GNN-AD-Navigator`, an end-to-end, deep learning-driven pipeline for automated Active Directory privilege escalation. By bridging the gap between offensive security data collection and advanced graph representation learning, the architecture successfully translated raw BloodHound json fragments into compiled PyTorch Geometric (PyG) tensors. Through the application of a Heterogeneous Graph Transformer (HGT) and a tailored heuristic beam search algorithm, the model demonstrated the capacity to autonomously navigate complex enterprise topologies, ultimately surfacing highly stealthy attack paths via a streamlined Streamlit graphical interface.

The research yielded three highly defensible operational contributions. First, the architecture demonstrated exceptional zero-shot generalisation. When deployed against the previously unseen, 4 000-node INLANEFREIGHT enterprise environment, the model successfully routed unprivileged footholds to DCSync capabilities across multi-domain trust boundaries. Second, the research exposed the "Ghost Edge" anomaly, proving that by operating directly on the unrolled mathematical tensors, the neural network could surface stealthy cross-domain relationships that the industry-standard BloodHound GUI completely failed to render due to presentation-layer heuristics. Finally, the evaluation highlighted the principle of pipeline dominance: the discovery that updating the Python ingestion pipeline to recognise novel edges (such as `AllowedToDelegate`) immediately unlocked new traversal capabilities without requiring the neural network weights to be retrained.

Crucially, this work maintains an honest appraisal of its operational scope. The system functions as a highly advanced proof-of-concept constrained by its training distribution. The 33 curated topological traces extracted from the GOAD environment define a strict coverage ceiling. The model operates with high precision within these bounds but exhibits documented failure states when pushed beyond them. Limitations such as beam search depth constraints, the heuristic pruning of Out-of-Distribution cross-forest edges, and terminal state blindness clearly illustrate the boundaries of stateless topological inference. The architecture is not broken; rather, it behaves exactly as its mathematical configuration dictates at the edge of its coverage.

Ultimately, these identified limitations provide a clear, structured roadmap for future research. The transition from stateless topology mapping to stateful, context-aware navigation via Reinforcement Learning represents the most significant avenue for

evolution. Furthermore, expanding the supervised training corpus to heavily weight ADCS certificate abuse, alongside hardening the data engineering pipeline to enforce strict cross-forest protocol boundaries, will directly address the current edge cases. This thesis does not represent a final, exhaustive product, but rather a robust, academically rigorous foundation for the next generation of AI-driven offensive security tooling.

Bibliography

[1] HackTheBox academy. "Introduction to active directory." (2023), [Online]. Available: https://academy.hackthebox.com/module/74/section/1347 (visited on 06/10/2026).

[2] Microsoft Corporation. "Active directory security architecture." (2023), [Online]. Available: https://learn.microsoft.com/en-us/windows-server/identity/ (visited on 06/21/2026).

[3] C. Neuman, T. Yu, S. Hartman, and K. Raeburn, *The kerberos network authentication service (v5)*, Internet Requests for Comments, RFC, 2005.

[4] W. Schroeder. "A guide to attacking domain trusts." (2017), [Online]. Available: https://www.harmj0y.net/blog/redteaming/a-guide-to-attacking-domain-trusts/ (visited on 06/22/2026).

[5] W. Schroeder and L. Christensen, "An ace up the sleeve: Designing active directory dacl backdoors," in *Black Hat USA*, Black Hat, 2017.

[6] B. Delpy and V. L. Toux. "Mimikatz: Dcsync and the drsr protocol." (2015), [Online]. Available: https://github.com/gentilkiwi/mimikatz (visited on 06/22/2026).

[7] W. Schroeder and L. Christensen, "Certified pre-owned: Abusing active directory certificate services," in *Black Hat USA*, Black Hat, 2021.

[8] E. Shamir. "Wagging the dog: Abusing resource-based constrained delegation to attack active directory." (2019), [Online]. Available: https://shenaniganslabs.io/2019/01/28/Wagging-the-Dog.html (visited on 06/22/2026).

[9] SpecterOps. "Bloodhound: Six degrees of domain admin." (2016), [Online]. Available: https://github.com/BloodHoundAD/BloodHound (visited on 06/22/2026).

[10] O. Lyak. "Certipy-ad: Active directory certificate services enumeration and abuse." (2022), [Online]. Available: https://pypi.org/project/certipy-ad/ (visited on 06/24/2026).

[11] D. jan Mollema. "Bloodhound-python: A python based ingestor for bloodhound." (2018), [Online]. Available: https://github.com/fox-it/bloodhound-python (visited on 06/22/2026).

[12] O. Lyak. "Certipy: Active directory certificate services enumeration and abuse." (2022), [Online]. Available: https://github.com/ly4k/Certipy (visited on 06/23/2026).

[13] T. Be'ery and T. Maor. "Gofetch: Automatically deploying privilege escalation tools." (2017), [Online]. Available: https://github.com/GoFetchAD/GoFetch (visited on 06/22/2026).

[14] V. L. Toux. "Pingcastle: Active directory security tool." (2017), [Online]. Available: https://www.pingcastle.com/ (visited on 06/22/2026).

[15] MDPI, "Physics-informed graph neural networks for attack path prediction in active directory," *Algorithms*, vol. 18, 2025. [Online]. Available: https://www.mdpi.com/2624-800X/5/2/15.

[16] Z. Hu, Y. Dong, K. Wang, and Y. Sun, "Heterogeneous graph transformer," in *Proceedings of The Web Conference 2020*, ACM, 2020, pp. 2704–2710.

[17] SpecterOps. "Bloodhound community edition v8 launches with opengraph." (2025), [Online]. Available: https://specterops.io/blog/2025/07/29/bloodhound-community-edition-v8-launches-with-opengraph-identity-attack-paths-beyond-active-directory-entra-id/ (visited on 06/22/2026).

[18] M. Boon. "Bloodhound on kali linux 2025: The guide i wish existed." (2025), [Online]. Available: https://medium.com/@mauriceboon88/bloodhound-plumhound-on-kali-linux-2025-the-guide-i-wish-existed-before-i-lost-2-hours-fighting-397e43035fa7 (visited on 06/22/2026).

[19] X. Sun and J. Yang, "Hetglm: Lateral movement detection by discovering anomalous links with heterogeneous graph neural network," in *2022 IEEE International Performance, Computing, and Communications Conference (IPCCC)*, IEEE, 2022. DOI: 10.1109/IPCCC55026.2022.9894347.

[20] Z. Hu, Y. Dong, K. Wang, and Y. Sun, "Heterogeneous graph transformer," in *Proceedings of the Web Conference 2020*, 2020, pp. 2704–2710.

Appendix A

Sanitised BloodHound JSON

The following snippet demonstrates the structural schema of the raw BloodHound JSON data before it is ingested and vectorised by the data pipeline. Extraneous metadata (such as timestamps, unpopulated arrays, and logon statistics) has been omitted for brevity.

This specific example highlights a user object (`drogon@essos.local`) and demonstrates how topological relationships (such as a `GenericAll` edge) are stored within the `Aces` array prior to being mapped into the `HeteroData` edge indices.

```
{
  "users": [
    {
      "ObjectIdentifier": "S
         -1-5-21-3939323223-2065505069-2611713634-1118",
      "Properties": {
        "name": "drogon@essos.local",
        "domain": "essos.local",
        "highvalue": false,
        "sensitive": false,
        "admincount": true,
        "unconstraineddelegation": false
      },
      "Aces": [
        {
          "PrincipalSID": "S
             -1-5-21-3939323223-2065505069-2611713634-512",
          "PrincipalType": "Group",
          "RightName": "GenericAll",
          "IsInherited": false
        }
      ],
      "HasMember": [],
      "MemberOf": []
    }
```

```
24    ]
25 }
```

Listing A.1: Truncated and sanitised JSON structure for a User node.

Appendix B

Heterogeneous Graph Schema

This appendix details the structural schema of the PyTorch Geometric (PyG) `HeteroData` object utilised by the `GNN-AD-Navigator`. The graph is inherently heterogeneous, comprising multiple distinct node and edge types derived from the merged Active Directory environment.

B.1 Node Schema

The compiled graph consists of 7 distinct node types. To ensure dimensional consistency across the Heterogeneous Graph Transformer (HGT) layers during message passing, every node type is embedded with a uniform feature vector of size 14.

Node Type	Node Count (GOAD)	Feature Dimension
`users`	48	14
`groups`	166	14
`computers`	5	14
`domains`	3	14
`ous`	12	14
`gpos`	8	14
`containers`	66	14

Table B.1: Node types, dataset distributions, and their corresponding feature dimensions within the PyG tensor.

B.2 Edge Schema

The HGT architecture processes directed relationships as distinct triplets in the format `(source_type, edge_type, target_type)`. Due to the mathematical permutation of Active Directory privileges across different objects (e.g., a `GenericAll` edge can originate from a user, a group, or a computer), the complete training dataset contains 75 unique edge triplets.

Table B.2 provides a representative sample of these critical transitions extracted directly from the `HeteroData` edge indices.

Source Node	**Edge / Attack Type**	**Target Node**
users	allowedtodelegate	computers
groups	genericall	users
groups	addkeycredentiallink	users
users	genericwrite	users
users	forcechangepassword	users
groups	readgmsapassword	users
computers	readgmsapassword	users
groups	owns	computers
groups	writedacl	users

Table B.2: A representative subset of the 75 directional edge triplets processed by the neural policy head.

Appendix C

Key Code Excerpts

Effective Weight Loss Function

```
# eff_weight combines path importance with step discretion cost
eff_weight = path_weight * (1 - step_cost)
```

Listing C.1: Effective weight computation in `prepare_training_examples.py`

DCSync Validation Logic

The `has_dcsync_on` function mathematically verifies if a node has acquired the requisite combination of rights (`GetChanges` and either `GetChangesAll` or `GetChangesInFilteredSet`) to perform a DCSync attack against the target domain.

```
def has_dcsync_on(node_idx, target_domain_name,
   edge_tensors_flat, idx_to_name):
    held_on = set()
    for rel in DCSYNC_RIGHTS:
        ei = edge_tensors_flat.get(rel)
        if ei is None: continue
        src, dst = ei
        mask = (src == node_idx)
        for d in dst[mask].tolist(): held_on.add((rel,
           idx_to_name.get(d, "")))

    has_getchanges    = {d for r, d in held_on if r == "
       getchanges"}
    has_getchangesall = {d for r, d in held_on if r == "
       getchangesall"}
    has_filtered      = {d for r, d in held_on if r == "
       getchangesinfilteredset"}

    domains_with_both = has_getchanges & (has_getchangesall |
       has_filtered)
```

```
    won = False
    for d in domains_with_both:
        if target_domain_name == d or target_domain_name.
           endswith("." + d):
            won = True; break
    return won, list(domains_with_both)
```

Listing C.2: Terminal state validation for DCSync execution in `inference.py`

Heuristic Beam Search Core Loop

This excerpt highlights the core navigation loop of the inference engine. It demonstrates the neural policy head scoring candidate neighbours, applying the Softmax activation, and the critical fallback bypass mechanism used to prevent the pruning of Out-of-Distribution (OOD) edges.

```
def beam_search(model, embeddings, edge_tensors_flat, start_idx
   , target_idx, idx_to_name, edge_name_map, known_edges,
   beam_width=3, max_depth=8):
    target_domain = get_node_domain(target_idx, idx_to_name) or
        ""
    beam = [(0.0, [start_idx])]
    completed = []

    for _ in range(max_depth):
        candidates = []
        for log_score, path in beam:
            cur = path[-1]
            won, _ = has_dcsync_on(cur, target_domain,
               edge_tensors_flat, idx_to_name)

            if cur == target_idx or won:
                completed.append((log_score, path))
                continue

            nbrs = get_neighbors(cur, edge_tensors_flat)
            nbrs = [n for n in nbrs if n not in path and n <
               embeddings.shape[0]]
            if not nbrs: continue

            with torch.no_grad():
                scores = model.score(embeddings, cur, nbrs)
```

```
                probs  = F.softmax(scores, dim=0)

            for i, nb in enumerate(nbrs):
                # 1. Identify the relationship type
                rel = edge_name_map.get((cur, nb), "unknown")

                # 2. THE FALLBACK BYPASS
                # If the model has never seen this edge during
                   training, bypass the
                # neural network's score and assign it a low
                   baseline operational score.
                if rel not in known_edges:
                    prob_val = 0.05
                else:
                    prob_val = probs[i].item()

                # 3. Calculate the new sequence score
                ns = log_score + math.log(prob_val + 1e-9)
                new_path = path + [nb]
                candidates.append((ns, new_path))

        if not candidates: break
        candidates.sort(key=lambda x: x[0], reverse=True)
        beam = candidates[:beam_width]
        if len(completed) >= beam_width: break

    # (Results sorting and trimming omitted for brevity)
    return trimmed
```

Listing C.3: The iterative beam search and scoring loop in `inference.py`

Appendix D

Interface and Visualisation

This appendix showcases the operational interfaces developed for the `GNN-AD-Navigator`. To ensure the underlying PyTorch Geometric models are accessible for practical security assessments, both a streamlined Command-Line Interface (CLI) and an interactive graphical web dashboard were engineered.

D.1 Command-Line Interface

The CLI is designed for headless server environments and automated scripting. It handles the entire pipeline—from data ingestion and tensor compilation to heuristic beam search inference—while logging deterministic audit results securely to the disk.

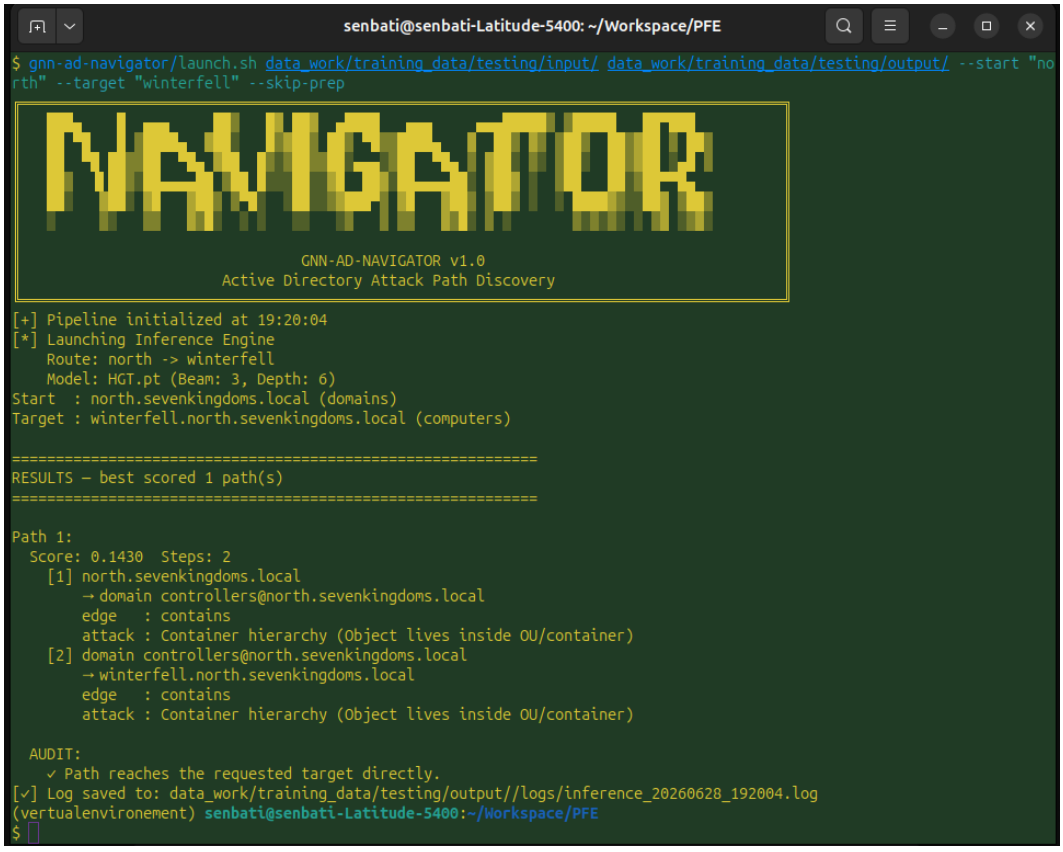

Figure D.1: The streamlined CLI executing a query

D.2 Streamlit Graphical Interface

For human-driven analysis, a Streamlit-based web interface provides dynamic visualisation of the HGT outputs. The dashboard allows operators to tune beam search parameters on the fly and interactively explore the generated attack paths via embedded `pyvis` network graphs.

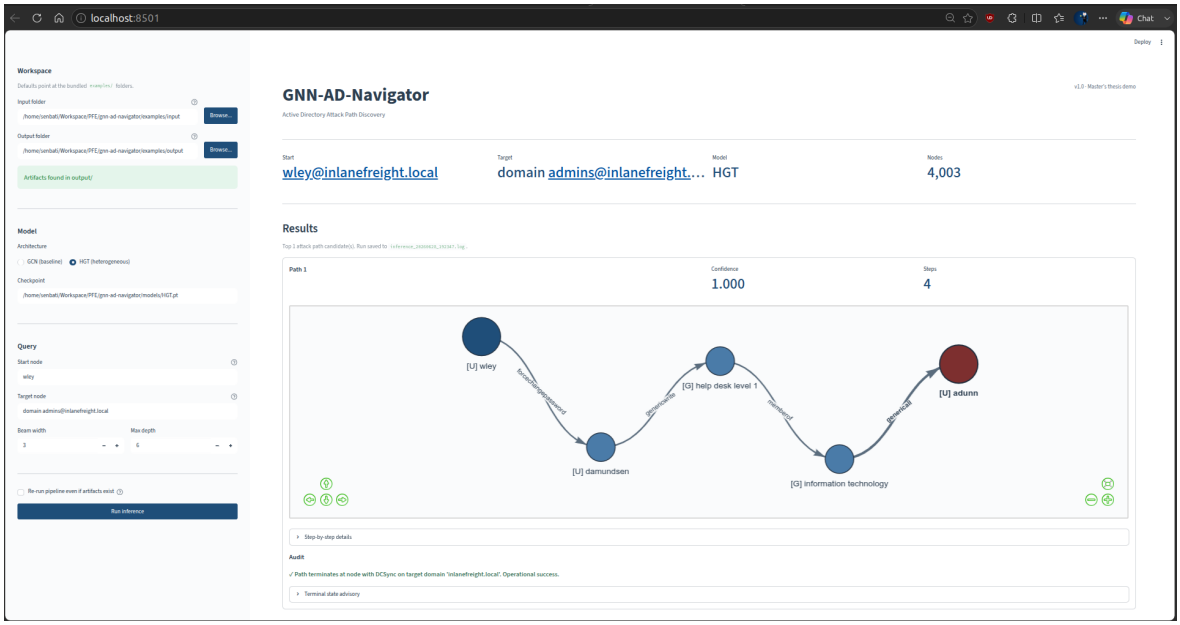

Figure D.2: The main dashboard rendering path from `wley` to Domain Admin in the 4 000-node INLANEFREIGHT environment.

Université Abdelmalek Essaâdi
Faculté des Sciences et Techniques de Tanger
Département Génie Informatique

Année Universitaire : 2025/2026

PV de Projet de Fin d'Etude (PFE)

Filière : Master en Sécurité It & Big Data

Nom et prénom de l'étudiant	Note Finale (Individuelle ou du Groupe)
SENBATI Nizar	

Sujet du projet :

Learning Offensive Navigation Policies on Heterogeneous Attack Graphs

A GNN Approach to Sequential Privilege Escalationin Active Directory

Date de la soutenance : 02/07/2026

Membres du Jury	Signature
Pr. Lotfi EL AACHAK ……: Président	
Pr Abdelhadi FENNAN ……: Examinateur	
Pr Abderrahim GHADI : ……Encadrant Interne	
Mr. ZAABOUL Jawad…… : Encadrant Externe	

Notes et Appréciations du jury :……

……

……

……